



COMPRESSÃO SELETIVA DE CONTEXTO VIA PROBLEMA DA MOCHILA:
UM *FRAMEWORK* NÃO INVASIVO PARA EXTENSÃO DA JANELA DE
CONTEXTO EM LLMS

Matheus Cardoso

Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação.

Orientadores: Geraldo Xexeo
Claudia Susie

Rio de Janeiro
Julho de 2026

COMPRESSÃO SELETIVA DE CONTEXTO VIA PROBLEMA DA MOCHILA:
UM *FRAMEWORK* NÃO INVASIVO PARA EXTENSÃO DA JANELA DE
CONTEXTO EM LLMS

Matheus Cardoso

DISSERTAÇÃO SUBMETIDA AO CORPO DOCENTE DO INSTITUTO
ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE
ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO
GRAU DE MESTRE EM CIÊNCIAS EM ENGENHARIA DE SISTEMAS E
COMPUTAÇÃO.

Orientadores: Geraldo Xexeo
Claudia Susie

Aprovada por: Prof. Geraldo Bonorino Xexeo
Profa. Claudia Susie Camargo Rodrigues
Prof. Nome do Examinador Externo

RIO DE JANEIRO, RJ – BRASIL
JULHO DE 2026

Cardoso, Matheus

Compressão Seletiva de Contexto via Problema da Mochila: um *Framework* Não Invasivo para Extensão da Janela de Contexto em LLMs/Matheus Cardoso. – Rio de Janeiro: UFRJ/COPPE, 2026.

XIII, 66 p.: il.; 29, 7cm.

Orientadores: Geraldo Xexeo

Claudia Susie

Dissertação (mestrado) – UFRJ/COPPE/Programa de Engenharia de Sistemas e Computação, 2026.

Referências Bibliográficas: p. 61 – 66.

1. Extensão de Contexto. 2. Compressão de *Prompt*. 3. Problema da Mochila. I. Xexeo, Geraldo *et al.* II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia de Sistemas e Computação. III. Título.

Agradecimentos

Agradeço, antes de tudo, a Deus, sem quem nada disso seria possível.

Agradeço à minha esposa, que me deu todo o suporte ao longo do mestrado. Sua presença foi essencial em cada etapa desta caminhada.

Agradeço aos meus pais, pelo apoio emocional e, sobretudo, pelo legado do estudo. Com eles aprendi que o conhecimento transforma vidas, e foi esse aprendizado que me trouxe até aqui.

Agradeço ao meu orientador, pelo apoio ao longo de todo o processo e por me instigar a extrair o meu melhor.

Agradeço, de forma póstuma, aos meus avós, que ouviram com paciência as reclamações de uma criança que dizia querer abandonar os estudos ao terminar o ensino fundamental, convencida de que estudar era insuportável. Foi graças a eles que criei gosto pelo aprender.

Por fim, agradeço à minha cachorrinha Jessie, que chegou há pouco às nossas vidas e já me ajudou a atravessar alguns momentos de ansiedade aqui na COPPE.

Resumo da Dissertação apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Mestre em Ciências (M.Sc.)

COMPRESSÃO SELETIVA DE CONTEXTO VIA PROBLEMA DA MOCHILA:
UM *FRAMEWORK* NÃO INVASIVO PARA EXTENSÃO DA JANELA DE
CONTEXTO EM LLMs

Matheus Cardoso

Julho/2026

Orientadores: Geraldo Xexéo
Claudia Susie

Programa: Engenharia de Sistemas e Computação

Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) tornaram-se componentes centrais de aplicações modernas, mas ainda esbarram em uma limitação estrutural, a janela de contexto. O mecanismo de atenção apresenta complexidade quadrática no comprimento da sequência e, mesmo quando a janela é nominalmente ampla, o modelo tende a não utilizar toda a informação de forma eficaz, com fenômenos como *lost in the middle*, *context rot* e colapso de atenção. Este trabalho investiga estratégias não invasivas de extensão de contexto, aplicáveis a modelos acessados via API, sem retreinamento nem modificação arquitetural, e propõe um *framework* que formula a compressão de *prompt* como um problema de alocação de orçamento. A partir de uma revisão sistemática das técnicas existentes e de uma avaliação empírica de cinco estratégias em *benchmarks* de contexto longo, o trabalho formaliza a seleção de compressores como um problema da mochila de múltipla escolha. Dado um *prompt* particionado, um orçamento efetivo de contexto e uma família de compressores parametrizados, o modelo escolhe, para cada partição, a opção de compressão que maximiza a informação preservada sob a restrição de orçamento. Uma triagem adicional compara LLMLingua-GPT2, LLMLingua-2, Selective Context e uma aproximação CPC-MiniLM em cinco modelos e dois orçamentos de contexto. CPC-MiniLM obteve a maior média entre os quatro compressores em oito das dez combinações, enquanto LLMLingua-2 liderou nas duas combinações do Qwen 3 30B-A3B. No Gemma 4 26B com 32768 *tokens*, CPC-MiniLM alcançou 0.573 e apresentou a maior média entre as nove técnicas comparadas. Esses resultados sustentam a seleção inicial de CPC-MiniLM, LLMLingua-2 e Selective Context

como opções do MCKP. A avaliação completa do *framework* proposto é apresentada como plano experimental reprodutível.

Abstract of Dissertation presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Master of Science (M.Sc.)

SELECTIVE CONTEXT COMPRESSION VIA THE KNAPSACK PROBLEM:
A NON-INVASIVE FRAMEWORK FOR CONTEXT WINDOW EXTENSION
IN LLMS

Matheus Cardoso

July/2026

Advisors: Geraldo Xexeo
Claudia Susie

Department: Systems Engineering and Computer Science

Large Language Models (LLMs) have become central components of modern applications, yet they still face a structural limitation, the context window. The attention mechanism has quadratic complexity in the sequence length and, even when the window is nominally large, the model tends not to use all of the available information effectively, exhibiting phenomena such as lost in the middle, context rot, and attention collapse. This work investigates non-invasive strategies for context extension, applicable to models accessed through an API, without retraining or architectural modification, and proposes a framework that formulates prompt compression as a budget allocation problem. Building on a systematic review of existing techniques and an empirical evaluation of five strategies on long-context benchmarks, the work formalizes compressor selection as a multiple-choice knapsack problem. Given a partitioned prompt, an effective context budget, and a family of parameterized compressors, the model selects, for each partition, the compression option that maximizes the preserved information under the budget constraint. An additional screening compares LLMLingua-GPT2, LLMLingua-2, Selective Context, and a CPC-MiniLM approximation across five models and two context budgets. CPC-MiniLM achieved the highest average among the four compressors in eight of the ten model-budget combinations, whereas LLMLingua-2 led in both Qwen 3 30B-A3B combinations. On Gemma 4 26B with 32,768 tokens, CPC-MiniLM reached 0.573 and achieved the highest average among all nine techniques. These results support the initial selection of CPC-MiniLM, LLMLingua-2, and Selective Context

as MCKP options. The complete evaluation of the proposed framework is presented as a reproducible experimental plan.

Sumário

Lista de Figuras	xii
Lista de Tabelas	xiii
1 Introdução	1
1.1 Contextualização	1
1.1.1 O Problema Crítico entre Janela de Contexto e Desempenho .	2
1.2 Problema de Pesquisa	3
1.3 Objetivos	4
1.3.1 Objetivo Global	4
1.3.2 Objetivos Específicos	4
1.4 Justificativa e Contribuições	5
1.4.1 Principais Contribuições	6
1.5 Estrutura da Dissertação	6
2 Fundamentos Técnicos e Teóricos	8
2.1 Fundamentos Técnicos de LLMs	8
2.1.1 Modelos de Linguagem de Grande Escala	8
2.1.2 Mecanismo de Atenção e Janela de Contexto	9
2.1.3 Complexidade Computacional Quadrática	9
2.2 Estratégias Não Invasivas para Extensão de Contexto	10
2.2.1 Compressão de Prompt	11
2.2.2 Segmentação de Contexto e Janela Deslizante	17
2.2.3 Métodos Baseados em MapReduce	17
2.2.4 Alocação e Gerenciamento de Memória Externa	18
2.2.5 Síntese e Taxonomia de Técnicas Não Invasivas	22
2.2.6 Avaliação de Eficácia, Métricas e Resultados	22
2.2.7 Benchmarks Especializados para Contexto Longo	22
2.2.8 Síntese e Lacunas Identificadas	24

3	Metodologia e <i>Framework</i> Proposto	25
3.1	Validação Empírica das Estratégias Não Invasivas	25
3.1.1	Objetivo e Desenho Experimental	25
3.1.2	Evolução do Protocolo Experimental	28
3.1.3	Fase Preliminar, Avaliação via APIs Externas	28
3.1.4	Resultados de Acurácia da Fase Preliminar	28
3.1.5	Resultados de Latência da Fase Preliminar	30
3.1.6	Discussão da Fase Preliminar	31
3.1.7	Revisão da Suíte de Benchmarks	31
3.1.8	Resultados da Matriz Local Ampliada	32
3.1.9	Seleção da Estratégia Base	38
3.2	<i>Framework</i> Proposto	39
3.2.1	Motivação e Visão Geral	39
3.2.2	Particionamento e Orçamento	39
3.2.3	Função de Importância	40
3.2.4	Opções de Compressão e Modelo de Qualidade	40
3.2.5	Restrições de Compatibilidade	41
3.2.6	Formulação como Extensão do Problema da Mochila de Múltipla Escolha	41
3.2.7	Solução por Programação Dinâmica	42
3.2.8	Algoritmo do <i>Framework</i>	42
3.2.9	Conjunto de Compressores Compatíveis	44
3.2.10	Implementação de Referência	45
3.2.11	Aplicação da Solução de Ponta a Ponta	48
3.3	Perguntas de Pesquisa e Hipóteses	48
3.4	Plano Experimental para Avaliação do <i>Framework</i>	49
4	Resultados e Discussão	50
4.1	Escopo da Evidência Experimental	50
4.2	Desempenho na Matriz Multimodelo	51
4.2.1	Efeito do Orçamento de Contexto	51
4.2.2	Variação entre Modelos e Tarefas	51
4.3	Relação entre Qualidade, Compressão e Latência	52
4.4	Análise dos Novos Compressores	52
4.5	Seleção das Opções do MCKP	56
4.6	Limitações da Análise	56
4.7	Síntese dos Resultados	57

5	Conclusão e Trabalhos Futuros	58
5.1	Síntese do Trabalho	58
5.2	Contribuições	59
5.3	Limitações	59
5.4	Trabalhos Futuros	60
	Referências Bibliográficas	61

Lista de Figuras

3.1	Arquitetura conceitual do <i>framework</i> proposto. O sistema transforma cada particao em alternativas com custo e qualidade, seleciona uma alternativa por particao sob o orcamento global e reconstrói o <i>prompt</i> antes da chamada ao modelo.	39
3.2	Fluxo proposto para resolver a selecao de compressores por programacao dinamica. A tabela registra a melhor qualidade acumulada para cada particao, orcamento parcial e ultima opcao; o retrocesso recupera as opcoes usadas na reconstrucao.	43
3.3	Sequencia conceitual da aplicacao do <i>framework</i> . Para cada particao, sao calculadas a importancia e as alternativas de compressao; o otimizador seleciona as alternativas globalmente antes da reconstrucao e da chamada ao modelo.	44

Lista de Tabelas

2.1	Comparação dos compressores de <i>hard prompt</i> considerados nesta dissertação	15
2.2	Taxonomia de técnicas não invasivas para extensão de contexto em LLMs	23
3.1	Modelos avaliados e diversidade arquitetural da matriz de <i>benchmarks</i>	27
3.2	Acurácia por estratégia, modelo e <i>benchmark</i> de contexto longo . . .	29
3.3	Latência média de resposta (ms) por estratégia, modelo e <i>benchmark</i>	30
3.4	Acurácia por técnica e <i>benchmark</i> nos cinco modelos, orçamento de 8192 <i>tokens</i> . As colunas abreviam LongBench (LB), ZeroSCROLLS (ZS), Natural Questions (NQ), TriviaQA (TQA), HotpotQA (HQA), MuSiQue (MSQ) e QMSum (QMS). O asterisco identifica a combinação com uma resposta excluída por <i>timeout</i>	33
3.5	Acurácia por técnica e <i>benchmark</i> nos cinco modelos, orçamento de 32768 <i>tokens</i> . As abreviações de coluna seguem a Tabela 3.4	34
3.6	Latência média (ms) e taxa de compressão efetiva por técnica nos cinco modelos. A taxa retida é a média das taxas nos dois orçamentos, em que 1.000 corresponde à ausência de compressão. O asterisco identifica a média de latência calculada após a exclusão de uma chamada encerrada por <i>timeout</i>	36
3.7	Baseline e compressores candidatos considerados para o MCKP . . .	45
4.1	Métricas operacionais dos quatro compressores nas dez combinações de modelo e orçamento	54

Capítulo 1

Introdução

1.1 Contextualização

Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) mudaram de forma decisiva todo o campo de processamento de linguagem natural (*Natural Language Processing*, NLP). Antes da popularização dos LLMs, grande parte das abordagens em NLP era baseada em *pipelines* rígidos, apoiados em técnicas como *tokenization*, *stemming*, *lemmatization*, TF-IDF e remoção de *stopwords*, que buscavam transformar texto bruto em representações numéricas esparsas, como *Bag-of-Words*, para então alimentar modelos estatísticos ou algoritmos clássicos de aprendizado de máquina. Essas abordagens dependiam fortemente de engenharia manual de atributos (*feature engineering*) e de ajuste fino (*fine-tuning*) específico para cada conjunto de dados, dificultando a generalização para novos domínios.

A introdução de representações distribuídas, como o Word2Vec em 2013 [1], começou a mudar o panorama ao permitir que modelos neurais aprendessem, de forma automática, vetores densos que capturavam as relações semânticas e sintáticas entre palavras, reduzindo a necessidade das etapas de normalização descritas anteriormente. No entanto, foi a adoção da arquitetura *Transformer*, proposta por Vaswani et al. em 2017 [2], que efetivamente reorganizou o campo. Modelos baseados nessa arquitetura, como o BERT [3] e o GPT [4], introduziram o paradigma de treinamento prévio com grandes volumes de dados de texto não estruturados, seguido de ajuste fino em tarefas específicas com conjuntos menores e rotulados. Esse paradigma de aprendizado por transferência (*transfer learning*) [5] permitiu que representações contextualizadas fossem aprendidas diretamente de textos comuns, elevando o desempenho em diversas tarefas.

Com a disponibilização comercial do ChatGPT em novembro de 2022, esses modelos deixaram de ser apenas objetos de pesquisa para se tornarem componentes centrais de sistemas que realizam tarefas de análise de sentimento, classificação de

textos, extração de informações específicas de documentos e apoio à programação [3, 6]. Modelos mais recentes, das famílias GPT, Claude, Gemini e LLaMA, são hoje integrados a assistentes virtuais, fluxos de análise e produção textual e ambientes de desenvolvimento de *software*, tornando-se parte de uma infraestrutura de base em aplicações modernas nas áreas financeira, da saúde, da programação e da educação [7, 8]. Apesar desse avanço, todos esses modelos ainda esbarram em uma limitação estrutural fundamental, a **janela de contexto**.

A janela de contexto corresponde ao número máximo de *tokens* que um modelo consegue processar como *input* em uma requisição, funcionando como uma memória de trabalho disponível durante a interação. À medida que esse limite aumenta, cresce o custo computacional, pois o mecanismo de atenção apresenta complexidade quadrática $O(n^2)$ no comprimento da sequência. Isso ocorre porque cada *token* é comparado com todos os demais da sequência, fazendo com que tanto o consumo de memória em GPUs quanto o tempo de processamento aumentem de forma acentuada conforme a janela de contexto se expande [2, 9]. Na prática, essa restrição impacta diretamente a viabilidade de aplicações que exigem interações longas com o usuário, como a análise de grandes documentos, a inspeção de grandes volumes de *logs* ou o processamento de múltiplos arquivos em uma única chamada.

Historicamente, houve um aumento expressivo no tamanho das janelas de contexto de modelos baseados em *Transformers*. Entre 2018 e 2019, o BERT apresentava janela de contexto de 512 *tokens*. Em 2023, o GPT-4 Turbo ampliou esse limite para 128.000 *tokens*. Em 2024, arquiteturas como Gemini 1.5 e Llama 3.1 romperam a barreira de um milhão de *tokens*. Nos lançamentos mais recentes, o foco parece ter migrado da ampliação da janela para a melhoria da qualidade e da eficiência das respostas por meio do aprimoramento do *reasoning* [10]. Essa expansão, embora expressiva, não eliminou os problemas a ela associados. Novos desafios surgiram. Além da complexidade computacional já mencionada, fenômenos como a degradação de desempenho com o aumento do contexto (*context rot*) [11] e o colapso da atenção (*rank-collapse*), em que as pontuações de atenção tendem à uniformidade, comprometem a eficácia dos modelos em cenários de contexto genuinamente extenso [12].

1.1.1 O Problema Crítico entre Janela de Contexto e Desempenho

Apesar da capacidade teórica de processar milhões de *tokens*, os modelos modernos frequentemente sofrem com alucinações decorrentes do uso ineficaz do contexto. Estudos recentes [11, 12] relatam a degradação do desempenho à medida que mais informação é inserida na janela de contexto. Essa limitação decorre de um recurso

de atenção finito, cujos retornos diminuem conforme novos *tokens* são adicionados. De forma análoga à memória humana, os LLMs enfrentam uma escassez de atenção intrínseca à sua arquitetura, em que cada *token* adicionado consome parte de um “orçamento de atenção”, dificultando a priorização das informações mais importantes.

Além disso, surgem desafios práticos e econômicos associados ao uso de contextos extensos, descritos a seguir.

- **Viés *lost in the middle*.** *Tokens* posicionados no início ou no final da sequência tendem a receber mais atenção, enquanto aqueles situados na região central tendem a ser ignorados. Liu et al. [13] mostram que o desempenho dos LLMs segue uma curva em formato de “U”, em que a informação no meio de grandes documentos é frequentemente desprezada, o que pode ocasionar alucinações.
- **Custo computacional e econômico.** Como o processamento de *tokens* é o principal componente de custo em servidores, o uso de contextos muito longos pode tornar determinadas aplicações economicamente inviáveis, exigindo técnicas de otimização como a compressão de *prompt* [14].
- **Latência.** Devido à complexidade $O(n^2)$ da atenção [2], contextos extensos aumentam drasticamente o tempo de *prefill* e de geração, elevando a latência em aplicações interativas [15].

1.2 Problema de Pesquisa

Os avanços nas arquiteturas e nas técnicas de extensão de contexto ainda não superam a limitação prática do uso de LLMs em cenários cotidianos. Os principais modelos, que são acessados por interface ou por API (como ChatGPT e Claude), impõem limites práticos ao comprimento da entrada e continuam sujeitos a alucinações, especialmente em contextos extensos. Para contornar essa limitação, tornou-se comum a adoção de mecanismos de memória externa e de geração aumentada por recuperação (*Retrieval-Augmented Generation*, RAG). Contudo, na prática, mecanismos de busca vetorial aproximam similaridade por distância em um espaço de *embeddings*, o que não garante, em todos os casos, que os trechos recuperados sejam os mais relevantes para a tarefa. Em Yu et al. [16] há uma discussão mais ampla sobre os limites das aplicações de RAG.

Um ponto pouco explorado é que a maioria dessas técnicas trata a compressão de forma **uniforme**, aplicando um único método, e frequentemente uma única taxa de compressão, a todo o *prompt*. Essa escolha desconsidera a estrutura interna do *prompt*. Instruções, perguntas, restrições de formato, evidências recuperadas,

exemplos e trechos de fundo não carregam o mesmo valor para a tarefa, nem toleram o mesmo grau de compressão. Uma taxa uniforme pode comprimir demais uma região crítica e comprimir de menos uma região redundante.

Diante desse cenário, este trabalho parte da seguinte pergunta central.

Como estender, de forma não invasiva, a quantidade de informação útil que um LLM acessado via API consegue aproveitar dentro de sua janela de contexto, selecionando e comprimindo seletivamente as regiões do prompt de modo a preservar a informação mais relevante para a tarefa sob uma restrição de orçamento de tokens?

Diferentemente de abordagens que modificam diretamente o mecanismo de atenção ou os *position embeddings*, este estudo propõe um *framework* que atua sobre a entrada do modelo. A ideia central é tratar a compressão de *prompt* não como uma redução uniforme de comprimento, mas como um problema de **alocação de orçamento**, distribuindo a pressão de compressão entre as regiões do *prompt* segundo seu valor informacional e seu risco de perda, e formulando a seleção de compressores como um problema da mochila de múltipla escolha.

1.3 Objetivos

1.3.1 Objetivo Global

Propor um *framework* não invasivo para extensão da janela de contexto em LLMs que formule a compressão de *prompt* como um problema de alocação de orçamento, mais especificamente um problema da mochila de múltipla escolha, selecionando, para cada região do *prompt*, a estratégia de compressão que maximiza a informação preservada sob uma restrição de orçamento de *tokens*, sem retreinamento nem modificação arquitetural do modelo base.

1.3.2 Objetivos Específicos

1. Revisar sistematicamente as estratégias contemporâneas de extensão de contexto que operam **sem** os três elementos a seguir.
 - (a) retreinamento do modelo;
 - (b) modificação dos mecanismos de atenção;
 - (c) alteração dos sistemas de *Position Embeddings*.
2. Avaliar empiricamente as estratégias não invasivas mais promissoras em *benchmarks* especializados em contexto longo, medindo acurácia e latência em modelos de diferentes escalas;

3. Formalizar a seleção de compressores por região do *prompt* como um problema da mochila de múltipla escolha, com restrições de orçamento e de compatibilidade entre técnicas;
4. Especificar um conjunto de compressores compatíveis (*text-in/text-out*) e uma função de importância por partição que combine relevância à tarefa, densidade de informação e saliência posicional;
5. Discutir métricas de avaliação que considerem não apenas o desempenho final, mas também a preservação de informação, a fidelidade semântica e o risco de alucinação por perda de conteúdo.

1.4 Justificativa e Contribuições

A pesquisa sobre extensão de contexto em LLMs tem se concentrado predominantemente em intervenções arquiteturais diretas. Trabalhos como [2, 6] estabeleceram o mecanismo de auto-atenção como elemento central para a captura de dependências de longo alcance, inspirando uma linha de pesquisa dedicada a modificações estruturais do *Transformer*. Diversos *surveys* consolidam essa ênfase arquitetural, categorizando técnicas que alongam o contexto por meio de ajustes no *Transformer*, otimizações do mecanismo de atenção e esquemas especializados de codificação posicional [9, 17–21]. Embora essas abordagens demonstrem ganhos teóricos significativos, sua aplicação prática frequentemente esbarra em limitações de acesso, pois a maioria dos usuários de LLMs interage com modelos via API ou interfaces de chat, sem controle sobre a arquitetura interna ou capacidade de retreinamento.

Em contraste, este trabalho adota estratégias **não invasivas**, isto é, técnicas aplicáveis sobre modelos de API sem necessidade de retreinamento, *fine-tuning* ou modificação de componentes arquiteturais. Essas estratégias incluem compressão de *prompts* [14, 22], segmentação e processamento sequencial de janelas de contexto [23], arquiteturas de memória externa persistente [24, 25] e recuperação aumentada de informações externas [16, 26, 27]. Trabalhos recentes têm refinado essas técnicas com abordagens como o ganho de informação relevante para seleção de trechos [28], a integração de grafos de conhecimento esparsos [29] e a otimização de memória em tempo de execução [30]. Choi et al. [31] demonstram, por exemplo, que a combinação de RAG com adaptação de baixo-*rank* (LoRA) pode melhorar a precisão de recuperação ao mesmo tempo em que reduz custos computacionais, evidenciando o potencial de técnicas não invasivas em aplicações especializadas.

Além disso, este trabalho propõe um diferencial metodológico. Em vez de tratar a compressão de *prompt* como redução uniforme de *tokens*, introduz-se a noção de

densificação informacional orientada à tarefa, na qual a compressão é entendida como a transformação de um contexto longo em uma representação menor que preserva a informação utilizável para a tarefa, sob restrições operacionais e epistêmicas. A partir dessa noção, a seleção do compressor de cada região do *prompt* é formulada como um problema da mochila de múltipla escolha, permitindo que o *framework* funcione como uma camada de decisão acima dos compressores já existentes.

1.4.1 Principais Contribuições

Este trabalho oferece as seguintes contribuições.

1. Uma revisão sistemática de técnicas **não invasivas** de extensão de contexto, categorizando abordagens aplicáveis sem acesso aos parâmetros ou à arquitetura interna do modelo, organizada em uma taxonomia por lógica operacional;
2. Uma avaliação empírica comparativa de cinco estratégias de extensão de contexto em quatro modelos de escalas distintas, reportando acurácia e latência em *benchmarks* de contexto longo;
3. Um *framework* de **compressão seletiva via problema da mochila**, que atua como uma camada de decisão acima de compressores existentes, alocando a pressão de compressão entre as regiões do *prompt* de acordo com seu valor informacional e seu risco de perda;
4. Um plano experimental reproduzível para a avaliação do *framework*, incluindo um conjunto de compressores compatíveis, *benchmarks*, *baselines* e estudos de ablação, orientado por métricas de preservação de informação e não apenas de desempenho final.

1.5 Estrutura da Dissertação

Esta dissertação está organizada em cinco capítulos, estruturados de forma a conduzir o leitor desde os fundamentos técnicos até a formulação e o plano de avaliação do *framework* proposto.

O **Capítulo 1** contextualiza o problema da janela de contexto em LLMs, apresenta os desafios práticos de custo, latência e degradação de desempenho em contextos extensos, formula a pergunta de pesquisa central, define os objetivos e apresenta as contribuições esperadas.

O **Capítulo 2** revisa os princípios técnicos dos LLMs, que abrangem a arquitetura *Transformer*, o mecanismo de atenção, a complexidade computacional e o

funcionamento da janela de contexto. Em seguida, discute as limitações práticas dos modelos em contextos longos, incluindo fenômenos como *context rot*, *lost in the middle* e colapso de atenção, e propõe uma taxonomia das estratégias não invasivas de extensão de contexto.

O **Capítulo 3** apresenta a validação empírica das técnicas não invasivas revisadas, com seleção da abordagem de melhor desempenho segundo *benchmarks* especializados, e formaliza a arquitetura do *framework* proposto, descrevendo a compressão seletiva de contexto como um problema da mochila de múltipla escolha, com sua função de importância, seu modelo de qualidade, suas restrições de compatibilidade e seu algoritmo de solução.

O **Capítulo 4** apresenta e discute os resultados obtidos na avaliação empírica das estratégias, analisa a relação entre acurácia e latência entre modelos e *benchmarks*, e posiciona o *framework* proposto em relação a esses resultados, detalhando as perguntas de pesquisa e as hipóteses que orientam sua avaliação.

O **Capítulo 5** sintetiza os principais achados, reafirma as contribuições e discute as implicações práticas e teóricas do *framework*. Identifica limitações metodológicas, desafios de escalabilidade e direções para investigações futuras, incluindo a execução completa do plano experimental, a integração com modelos multimodais e a otimização de latência.

Capítulo 2

Fundamentos Técnicos e Teóricos

Este capítulo revisa os fundamentos técnicos necessários à compreensão do problema tratado nesta dissertação e organiza a literatura de extensão de contexto não invasiva em uma taxonomia. A Seção 2.1 apresenta os princípios dos LLMs, o mecanismo de atenção e a origem da complexidade quadrática. A Seção 2.2 revisa e classifica as principais estratégias de extensão de contexto que operam sem modificação do modelo base.

2.1 Fundamentos Técnicos de LLMs

2.1.1 Modelos de Linguagem de Grande Escala

LLMs podem ser classificados como redes neurais profundas, baseadas na arquitetura *Transformer* [2], treinadas para prever o próximo *token* em um grande *corpus* de texto. Matematicamente, esses modelos aproximam uma distribuição condicional $p(x_t | x_{<t})$, onde x_t representa o *token* atual e $x_{<t}$ denota todos os *tokens* anteriores da sequência [2, 6].

Diferentemente de modelos de rede neural especializados, que necessitam de treinamento e ajuste fino para realizar uma tarefa específica, os LLMs funcionam como plataformas de propósito geral em que o próprio usuário, em linguagem natural, especifica o comportamento e a ação desejada, como resumir documentos, responder perguntas, traduzir textos, analisar dados ou gerar código [6, 7, 18]. Essa capacidade de generalização sem exemplos rotulados é denominada *zero-shot* ou *few-shot*. Isso é possível em razão do alto volume e da variação das fontes de dados utilizados no treinamento dos modelos. Esses dados provêm de diversas origens, como internet, livros, códigos e artigos, permitindo ao modelo capturar padrões linguísticos, estruturas semânticas e conhecimento contidos nesses documentos. A qualidade e a diversidade dos dados de treinamento determinam fundamentalmente a capacidade desses modelos generalistas em realizar tarefas específicas sem supervisão.

2.1.2 Mecanismo de Atenção e Janela de Contexto

O mecanismo central dos LLMs é a **atenção multi-cabeças**, proposta por Vaswani et al. [2]. Seu funcionamento consiste no seguinte. Para cada *token* de entrada, o mecanismo calcula pontuações de atenção que quantificam o quanto esse *token* é importante para os demais, permitindo capturar dependências textuais de forma paralela.

Do ponto de vista matemático, cada *token* da sequência é projetado em três partes, a Consulta Q , a Chave K e o Valor V . A auto-atenção pode ser abstraída como um mecanismo que mede a similaridade entre a consulta e as chaves e que, em seguida, usa essas pontuações como pesos para combinar os valores correspondentes. Formalmente, o mecanismo de atenção pode ser representado pela Equação 2.1.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2.1)$$

O termo $QK^\top$ produz uma matriz em que cada entrada representa a similaridade (via produto escalar) entre a consulta de um *token* e a chave de outro. Com isso, cada linha dessa matriz indica o quanto um *token* considera relevantes os demais. O fator $\sqrt{d_k}$, onde d_k é a dimensão dos vetores de chave, é usado para normalizar esses produtos escalares e evitar valores muito grandes, que poderiam saturar a função de ativação e dificultar o treinamento [2]. Em seguida, a função *softmax* é aplicada a cada linha dessa matriz de similaridade para transformá-la em uma distribuição de probabilidades, de modo que os pesos de atenção de um *token* sobre todos os outros somam 1. Por fim, essa matriz de probabilidades é multiplicada por V , de modo que cada *token* receba uma combinação linear dos vetores de valor de todos os *tokens*, ponderada pela importância atribuída a cada um. O resultado é uma nova representação em que um *token* incorpora, de forma explícita, informação dos demais *tokens* considerados relevantes.

2.1.3 Complexidade Computacional Quadrática

Conforme mencionado na Seção 2.1.2, o mecanismo de auto-atenção apresenta complexidade $O(n^2 \cdot d)$. Essa complexidade decorre diretamente do cálculo da matriz de similaridade. Do ponto de vista de implementação, essa matriz de atenção $n \times n$ precisa ser armazenada em memória durante o cálculo, e cada camada do modelo (tipicamente entre 24 e 96 camadas em modelos modernos) mantém seu próprio *cache* de chaves e valores (*KV cache*) para evitar reproprocessamento durante a geração autorregressiva. O consumo total de memória cresce não apenas com n^2 , mas também com o número de camadas e o tamanho dos *batches* processados simultaneamente [30, 32].

As implicações práticas dessa complexidade incluem os pontos a seguir.

- **Limitação de memória em GPU.** Em modelos com contexto muito longo, o armazenamento simultâneo das matrizes de atenção ao longo de todas as camadas pode rapidamente consumir a maior parte da memória disponível, exigindo estratégias como particionamento de *cache*, paginação de memória (*PagedAttention*) e gerenciamento fino de alocação para viabilizar o funcionamento de LLMs em servidores de produção [30].
- **Latência em inferência.** O tempo de processamento do *prefill* cresce de forma quadrática com n . Em cenários de contexto muito longo, essa etapa pode prejudicar a latência do modelo, elevando o tempo de resposta e degradando a experiência do usuário.
- **Custo operacional.** Em ambientes de produção com precificação por *token*, processar repetidamente grandes quantidades de contexto eleva os custos financeiros da aplicação. Isso motiva o uso de estratégias de compressão de *prompt*, seleção de trechos importantes e recuperação seletiva da informação, que reduzem a necessidade de reprocessar interações [14].

Esses fatores reforçam a necessidade de buscar novas abordagens para processar o conteúdo da janela de contexto de forma mais eficiente. Como o objetivo deste trabalho é desenvolver um *framework* agnóstico a modelos de LLM produtivos de mercado, sem necessidade de retreinamento ou implementação de arquiteturas customizadas, a seção seguinte revisa sistematicamente as estratégias não invasivas, que operam sobre modelos já treinados sem modificações arquiteturais.

2.2 Estratégias Não Invasivas para Extensão de Contexto

Em razão do problema de aumento de *tokens* na janela de contexto e do fato de que os novos modelos não estão evoluindo exclusivamente no aumento da quantidade de *tokens* de entrada, uma variedade de técnicas não invasivas vem surgindo para contornar essas restrições. Definem-se como **técnicas não invasivas** aquelas que operam **sem modificação do modelo base**, ou seja, sem retreinamento, sem acesso aos parâmetros internos e sem modificação das matrizes internas de peso e posição. Essa restrição visa permitir a aplicação em modelos acessados via API e comercialmente mais utilizados, como ChatGPT, Claude, Gemini ou Grok. Na literatura, poucos trabalhos tratam especificamente do aumento de janela de contexto não invasivo [9, 17]. Cabe registrar que algumas abordagens estendem a janela sem

ajuste fino, mas ainda intervêm na computação da atenção, como o Self-Extend, que combina atenção agrupada e atenção de vizinhança para extrapolar o comprimento sem treinamento [33]. Como essa intervenção exige acesso ao mecanismo interno de atenção, tais métodos não se aplicam a modelos disponibilizados apenas via API e ficam fora do escopo aqui adotado.

Esta seção revisa as principais técnicas de aumento da janela de contexto e elabora uma proposta de taxonomia das principais estratégias identificadas na literatura. Essa classificação é organizada em quatro categorias principais conforme sua lógica operacional, sendo elas a compressão de *prompt*, a segmentação de contexto e janela deslizante, os métodos baseados em *MapReduce* e a alocação e gerenciamento de memória externa.

2.2.1 Compressão de Prompt

A **compressão de *prompt*** reduz a entrada entregue ao modelo alvo e procura preservar a informação necessária à tarefa. Revisões recentes organizam a área em métodos de *prompt* rígido (*hard prompt*) e contínuo (*soft prompt*) [34, 35]. Os primeiros selecionam unidades discretas do texto e produzem uma saída textual, enquanto os segundos substituem parte do contexto por representações vetoriais. Essa distinção se refere à representação produzida, e não à necessidade de treinamento. LLMingua-2 e CPC, por exemplo, geram texto comprimido, mas dependem de modelos auxiliares previamente treinados [36, 37]. Ainda assim, ambos preservam o modelo alvo sem alteração e podem ser empregados com LLMs acessados por API. Essa propriedade torna os métodos de *hard prompt* particularmente relevantes para o escopo não invasivo desta dissertação.

Seja x um *prompt* com L *tokens* e $\tilde{x}$ sua versão comprimida com $\tilde{L}$ *tokens*. A taxa de retenção é definida por $\tau = \tilde{L}/L$, enquanto a razão de compressão corresponde a $1/\tau$. O problema pode ser interpretado como a busca por $\tilde{x}$ que reduz τ e mantém a distribuição das respostas do modelo próxima àquela induzida por x [14]. Essa formulação evidencia que a redução de comprimento não constitui, isoladamente, uma medida de qualidade. Uma avaliação adequada também deve considerar o desempenho na tarefa, a fidelidade, a latência do compressor e o custo total de inferência.

LLMLingua. Jiang et al. [14] propõem uma arquitetura de compressão do geral para o específico (*coarse-to-fine*) baseada na perplexidade estimada por um modelo de linguagem menor. O controlador de orçamento distingue instrução, demonstrações e pergunta, preservando uma fração maior da instrução e da pergunta, diretamente associadas à definição da tarefa, e atribuindo maior compressão ao conjunto

potencialmente redundante de demonstrações. Sob restrições mais severas, demonstrações inteiras podem ser removidas antes da etapa fina. Em seguida, o algoritmo iterativo de compressão em nível de *token* processa o texto em segmentos e condiciona a análise de cada segmento ao prefixo já comprimido. Tokens com maior perplexidade são preservados, pois são tratados como menos previsíveis e, portanto, mais informativos para o modelo. Um estágio opcional de alinhamento ajusta o modelo auxiliar com textos gerados pelo LLM alvo, procurando reduzir a diferença entre as distribuições dos dois modelos. A formulação admite modelos causais de diferentes portes, incluindo GPT-2 e LLaMA. Os experimentos do artigo empregam principalmente Alpaca-7B e uma variante GPT2-small ajustada no conjunto Alpaca, evidenciando que a escolha do modelo auxiliar é parte da configuração experimental, e não a definição do método.

O método foi avaliado em raciocínio matemático, tarefas do BIG-Bench Hard, conversação e sumarização, por meio de GSM8K, BBH, ShareGPT e Arxiv-March23. Nos experimentos com GSM8K, a configuração mais agressiva reteve aproximadamente um vigésimo do *prompt* e apresentou redução de 1,52 ponto em correspondência exata em relação ao *prompt* completo. A análise de latência em uma GPU V100 reportou aceleração de ponta a ponta entre 1,7 e 5,7 vezes para razões de compressão entre 2 e 10 vezes [14]. Esses valores caracterizam as configurações do artigo e não devem ser generalizados diretamente para outros modelos ou infraestruturas. Os próprios autores observam degradação acentuada em GSM8K entre 25 e 30 vezes de compressão. Além disso, o desempenho depende da proximidade entre o modelo auxiliar e o LLM alvo, e a remoção de *tokens* pode produzir texto de difícil leitura humana. Por anteceder os demais métodos considerados e aparecer como comparador recorrente nos trabalhos posteriores, LLMLingua é adotado nesta dissertação como baseline de compressão.

Selective Context. Li et al. [38] propõem um compressor sem treinamento específico que estima a auto-informação de cada *token* por meio de um modelo causal. Os valores podem ser agregados em frases nominais ou sentenças, e as unidades com auto-informação abaixo de um percentil definido são removidas. A seleção é agnóstica à pergunta e se fundamenta na hipótese de que conteúdo facilmente previsto pelo modelo auxiliar contém maior redundância. A avaliação dos autores abrange reconstrução de contexto, sumarização, resposta a perguntas e continuação de conversas sobre artigos do arXiv, notícias da BBC e diálogos do ShareGPT. A granularidade de frase nominal apresentou os resultados mais estáveis, enquanto a remoção de sentenças foi mais sensível à configuração.

Com redução de 50% do contexto, o estudo reporta diminuição de 36% no uso de memória de inferência e de 32% no tempo por *token*, acompanhada por queda de

0,023 em BERTScore-F1. Na análise manual de fidelidade para resposta a perguntas, 3,8% das tuplas extraídas das respostas comprimidas não foram sustentadas pelas respostas de referência nessa mesma taxa [38]. A interpretação desses resultados requer cautela, pois grande parte da avaliação toma como referência a resposta produzida pelo próprio modelo com o contexto integral, e os textos foram limitados a 2048 *tokens*. Os autores também identificam dependência do segmentador de frases nominais e variação do percentil ótimo entre tarefas e contextos. Para o MCKP, o método oferece uma alternativa de baixo acoplamento e sem treinamento adicional, mas sua medida de surpresa não representa diretamente a relevância de uma unidade para a consulta.

LLMLingua-2. Pan et al. [36] questionam o uso da entropia de modelos causais como critério de compressão, pois esse sinal considera apenas o contexto anterior e não é aprendido especificamente para decidir quais palavras preservar. LLMLingua-2 reformula a compressão como classificação binária de palavras. O conjunto de treinamento é construído por destilação, mediante compressões extrativas produzidas pelo GPT-4 a partir de transcrições do MeetingBank. Os textos são divididos em blocos de até 512 *tokens*, as palavras preservadas são alinhadas ao original e exemplos com maior taxa de variação lexical ou maior lacuna de alinhamento são filtrados. Um codificador bidirecional, XLM-RoBERTa-large na configuração principal e mBERT na versão menor, aprende a probabilidade de preservação. Na inferência, as palavras com maior probabilidade são selecionadas até o orçamento desejado, mantendo-se sua ordem original e a integridade de palavras divididas em múltiplos *subtokens*.

O artigo avalia generalização dentro e fora do domínio por meio de MeetingBank, LongBench, ZeroSCROLLS, GSM8K e BBH. Sob restrições de 2000 e 3000 *tokens*, LLMLingua-2 superou LLMLingua e Selective Context na média reportada para LongBench e ZeroSCROLLS, embora tenha permanecido abaixo de LongLLMLingua, que utiliza a pergunta, em parte das tarefas de contexto longo. Em uma V100, o compressor foi de três a seis vezes mais rápido que os baselines de compressão avaliados e produziu aceleração de ponta a ponta entre 1,6 e 2,9 vezes para razões entre 2 e 5 vezes [36]. A principal limitação reconhecida pelos autores é a concentração da destilação em transcrições do MeetingBank. Os testes fora do domínio reduzem essa preocupação, mas não eliminam a possibilidade de mudança de distribuição. Sua natureza agnóstica à tarefa também evita recompressão por consulta, ao custo de não explorar relevância específica para a pergunta.

Context-aware Prompt Compression. Liskavets et al. [37] apresentam o Context-aware Prompt Compression (CPC, sigla adotada no artigo), que remove sentenças completas segundo sua relevância para a pergunta. O método constrói o

conjunto Context-aware Question-Relevance a partir de textos do WikiText. Pares sintéticos de pergunta e resposta são gerados para sentenças positivas, verificados para exigir informação distribuída no contexto e associados a sentenças negativas filtradas por similaridade semântica e pelo efeito de sua remoção sobre a probabilidade da resposta. Um Mistral-7B-Instruct é então ajustado com LoRA usando uma perda contrastiva entre pergunta, sentenças positivas e negativas, combinada a uma perda de predição de *tokens* mascarados para aprender representações bidirecionais. Durante a inferência, cada sentença recebe uma similaridade em relação à pergunta, e as mais relevantes são selecionadas até o orçamento de *tokens*, com restauração da ordem original.

Nos resultados publicados, CPC obteve as maiores médias entre os métodos comparados no LongBench sob limites de 2000 e 3000 *tokens*, além de resultados superiores nos conjuntos específicos Krapivin, PubMed, MeetingBank e SummScreen. O custo do compressor foi substancialmente menor que o de LLMingua e LongLLMingua. Entretanto, LLMingua-2 permaneceu ligeiramente mais rápido na mesma avaliação de latência, com médias de 0,23 segundo para LLMingua-2 e 0,28 segundo para CPC por amostra do LongBench [37]. O artigo não apresenta uma seção específica de limitações. Para esta dissertação, três implicações decorrem da arquitetura. A seleção precisa ser refeita para cada pergunta, o codificador Mistral-7B tem custo de memória superior ao dos codificadores empregados pelo LLMingua-2 e a granularidade de sentença preserva legibilidade, mas não remove redundância interna das sentenças selecionadas. Essas propriedades tornam CPC complementar, e não estritamente substituto, aos métodos em nível de palavra ou frase.

A Tabela 2.1 sintetiza as diferenças que afetam a composição do conjunto de opções do MCKP.

Com base nessa complementaridade, LLMingua permanece como baseline externo, enquanto LLMingua-2, Selective Context e CPC formam o conjunto de três candidatos à composição das opções do MCKP. O nome *llm-lingua*, por sua vez, designa o pacote oficial que reúne implementações de métodos da família, incluindo LLMingua e LLMingua-2, e não um quarto algoritmo de compressão independente [39]. A escolha cobre decisões em nível de palavra, unidade lexical e sentença, além de incluir critérios agnósticos e condicionados à pergunta. Essa recomendação decorre da diversidade operacional dos métodos, e não de uma conclusão antecipada de superioridade. A decisão experimental deve considerar conjuntamente qualidade na tarefa, fidelidade, taxa efetiva de compressão, latência e falhas de execução nos benchmarks descritos no Capítulo 3.

Tabela 2.1: Comparação dos compressores de *hard prompt* considerados nesta dissertação

Método	Unidade	Usa a pergunta	Modelo auxiliar	Compromisso principal
LLMLingua [14]	Demonstração e <i>token</i>	Não diretamente	Modelo causal, com alinhamento opcional	Baseline consolidado, mas com maior latência e baixa legibilidade
Selective Context [38]	<i>Token</i> , frase ou sentença	Não	Modelo causal sem treino específico	Baixo acoplamento, mas limiar e segmentação sensíveis ao contexto
LLMLingua-2 [36]	Palavra	Não	Codificador treinado por destilação	Baixa latência, mas dependência dos dados de destilação
CPC [37]	Sentença	Sim	Codificador ajustado contrastivamente	Preserva legibilidade e relevância, com custo maior e recompressão por pergunta

Métodos de prompt contínuo. Uma linha alternativa substitui trechos do contexto por vetores aprendidos, em geral obtidos por meio de treinamento ou ajuste do modelo. O Gisting treina o modelo para condensar instruções em um conjunto reduzido de *gist tokens* reutilizáveis [40]. O AutoCompressor adapta o modelo para gerar vetores de sumarização que condensam contextos longos como *soft prompts* [41]. O ICAE emprega um autoencoder de contexto para produzir representações compactas interpretáveis pelo modelo alvo [42]. O SoftPromptComp combina sumarização em linguagem natural com *soft prompts* treináveis [43], e o LLoCO aprende contextos longos de forma *offline* combinados a adaptação de baixo *rank* [44]. Por dependerem de treinamento, esses métodos alcançam taxas de compressão elevadas, mas não satisfazem a restrição de não invasividade adotada neste trabalho.

Compressão orientada a recuperação. No contexto de geração aumentada por recuperação, alguns métodos comprimem os documentos recuperados antes de fornecê-los ao modelo. O RECOMP propõe compressores extrativo e abstrativo para condensar múltiplos documentos recuperados [45]. O COCOM comprime os documentos em conjuntos de *embeddings* de contexto entregues ao decodificador [46], e o PISCO aplica destilação em nível de sentença para obter compressão elevada com pouca perda de acurácia [47]. Assim como os métodos de *prompt* contínuo, essas abordagens dependem de treinamento dos compressores.

Compressão em linguagem natural. Uma vertente busca reescrever o contexto em linguagem natural mais condensada, preservando a legibilidade do *prompt*. Gilbert et al. [22] comparam três abordagens de compressão, sendo a primeira básica (remoção de *stopwords*), a segunda semântica (seleção orientada por *embeddings*) e a terceira *lossless* (compressão sem perda informacional), e relatam que a compressão semântica, orientada apenas por instruções de *prompt*, sem modelo de compressão adicional, alcança reduções similares com ganhos em eficiência computacional. O Nano-Capsulator, por sua vez, aprende a comprimir *prompts* em um formato de linguagem natural transferível entre modelos [48].

Os benefícios práticos da compressão de *prompt* incluem os pontos a seguir.

- **Redução de custos.** Diminui custos operacionais proporcionalmente ao número de *tokens* removidos, particularmente relevante em APIs com precificação por *token*.
- **Manutenção de qualidade.** Preserva a semântica por meio da seleção de *tokens* informativos [14].
- **Adequação a respostas rápidas.** Reduz a carga computacional no *prefill*, sendo apropriada para aplicações que demandam baixa latência.

2.2.2 Segmentação de Contexto e Janela Deslizante

A **segmentação de contexto** baseia-se na divisão de textos longos em segmentos menores que cabem individualmente na janela de contexto do modelo, processados em paralelo ou sequencialmente com estratégias de janelas deslizantes (*sliding windows*) [49].

Essa abordagem envolve quatro etapas principais, descritas a seguir.

1. **Particionamento.** Divisão do documento longo em trechos com tamanho adequado à janela de contexto disponível.
2. **Processamento paralelo ou sequencial.** Cada segmento é processado independentemente ou em sequência, dependendo dos recursos disponíveis.
3. **Preservação de continuidade.** A utilização de janelas sobrepostas (*overlapping windows*) ajuda a preservar a continuidade semântica entre segmentos sucessivos, evitando perda de informação nas fronteiras.
4. **Agregação.** Os resultados de cada segmento são combinados em uma resposta unificada, preservando a coerência global.

A eficácia dessa abordagem depende criticamente de dois fatores, a correta definição dos limites dos segmentos, de modo a não quebrar dependências semânticas ou sintáticas importantes, e uma estratégia adequada de agregação dos resultados [49]. Ratner et al. [49] propõem o uso de **janelas paralelas de contexto**, em que múltiplos segmentos são processados simultaneamente e seus resultados combinados por mecanismos de votação ponderada ou fusão de representações.

Entre os benefícios dessa abordagem estão os pontos a seguir.

- **Inclusão de maior volume de dados.** Permite o processamento de textos que excedem significativamente a janela nominal do modelo.
- **Redução de custos.** Em alguns cenários, pode reduzir custos ao permitir que apenas segmentos relevantes sejam processados em detalhe.
- **Adequação a textos multi-documento.** Beneficia tarefas em que a informação relevante está distribuída entre múltiplos documentos com conteúdo potencialmente redundante.

2.2.3 Métodos Baseados em MapReduce

Os **métodos MapReduce** fragmentam o texto em segmentos (fase *Map*), processam cada segmento individualmente e, em seguida, agregam os resultados intermediários (fase *Reduce*), seguindo um paradigma de computação distribuída aplicado ao

processamento de linguagem natural [23]. Kamradt et al. [23] descrevem e avaliam três variações principais dessa abordagem, apresentadas a seguir.

1. **Reduce Simple.** Agregação contínua de resumos e segmentos processados, concatenando resultados de forma sequencial. Cada segmento é sumarizado, e os resumos são progressivamente combinados até que caibam na janela de contexto final.
2. **QA Rank.** Utiliza perguntas e respostas (*question answering*) para identificar os pontos principais de cada segmento que serão passados adiante na agregação, priorizando informação relevante à tarefa específica.
3. **Refine.** Agregação de resultados de forma hierárquica, em que *outputs* intermediários são refinados iterativamente até a convergência.

Esses métodos oferecem uma estrutura modular para processamento e agregação de informações, aumentando a precisão em tarefas de síntese e análise de textos extensos [23]. Entre os benefícios práticos estão os pontos a seguir.

- **Modularidade.** Permite compor de forma flexível as etapas de processamento, facilitando a customização para diferentes domínios e tarefas.
- **Escalabilidade.** Pode ser aplicado recursivamente a textos arbitrariamente longos, sem limite teórico de comprimento.
- **Precisão em síntese.** É particularmente eficaz em tarefas de sumarização multi-documento e análise comparativa, em que informações complementares devem ser integradas.

2.2.4 Alocação e Gerenciamento de Memória Externa

A **memória externa** oferece uma abordagem fundamentalmente distinta das anteriores. Em vez de tentar processar todo o conteúdo dentro da janela de contexto (comprimindo, segmentando ou agregando), o sistema mantém um repositório persistente de informações, como histórico de diálogos, documentos indexados ou grafos de conhecimento estruturados, e recupera dinamicamente apenas a informação mais relevante para cada consulta específica. Esta categoria engloba diversas técnicas complementares que compartilham o princípio de externalizar o armazenamento de contexto e recuperá-lo sob demanda, detalhadas a seguir.

MemGPT como Sistema Operacional. O MemGPT [24] integra LLMs a capacidades de memória hierárquica e gerenciamento de tarefas, fazendo com que o modelo funcione de forma análoga a um sistema operacional autônomo capaz de coordenar operações computacionais complexas. A abordagem caracteriza-se por três componentes principais, descritos a seguir.

- **Memória hierárquica.** Distinção explícita entre memória de curto prazo, que corresponde à janela de contexto ativa do modelo, e memória de longo prazo, que corresponde ao armazenamento persistente externo, tipicamente em banco de dados vetorial.
- **Gerenciamento automático.** O sistema determina dinamicamente quais informações manter em contexto ativo e quais arquivar na memória de longo prazo, utilizando heurísticas de relevância e recência.
- **Recuperação sob demanda.** Busca de informações históricas baseada em similaridade semântica, por meio de *embeddings*, e em critérios de relevância contextual.

A arquitetura do MemGPT inspira-se em sistemas operacionais tradicionais, tratando a janela de contexto como análoga à memória RAM, limitada e rápida, e o armazenamento externo como análogo ao disco, ilimitado e mais lento. O sistema decide autonomamente quando fazer a paginação de informações entre os dois níveis [24].

InfLLM e a Memória de Contexto Eficiente. O InfLLM [25] propõe um método de extrapolação de contexto longo sem treinamento adicional, baseado em uma memória de contexto eficiente que organiza e armazena informações de maneira escalável. A abordagem distingue-se por três características, descritas a seguir.

- **Armazenamento estruturado.** As informações do contexto são segmentadas e indexadas em estruturas de dados otimizadas para busca rápida.
- **Recuperação seletiva.** Apenas os segmentos mais relevantes para a consulta atual são injetados na janela de contexto, reduzindo a carga computacional.
- **Manutenção de coerência.** Mecanismos de controle garantem que dependências de longo alcance sejam preservadas mesmo quando informações intermediárias não estão ativas no contexto.

Dessa forma, o InfLLM possibilita que o modelo mantenha coerência e qualidade na geração de texto mesmo ao processar entradas extremamente longas, sem necessidade de retreinamento ou modificação arquitetural [25].

Relevant Information Gain (RIG). Pickett et al. [28] propõem uma melhoria nos métodos de RAG (*Retrieval-Augmented Generation*) ao incorporar o conceito de **ganho de informação relevante** para otimizar a seleção e integração dos dados recuperados. A abordagem tradicional de RAG recupera documentos baseando-se apenas em similaridade semântica, o que não garante que os trechos recuperados sejam os mais informativos para responder à consulta. O RIG introduz uma métrica que quantifica o quanto cada documento recuperado contribui para reduzir a incerteza sobre a resposta correta, priorizando trechos verdadeiramente informativos. Formalmente, dado um conjunto de documentos candidatos $\{d_1, d_2, \dots, d_k\}$, o RIG é calculado conforme a Equação 2.2.

$$\text{RIG}(d_i) = H(Y) - H(Y \mid d_i), \quad (2.2)$$

onde $H(Y)$ é a entropia da distribuição de respostas antes de observar o documento, e $H(Y \mid d_i)$ é a entropia condicional após observar d_i . Documentos que maximizam o ganho de informação são prioritariamente incluídos no contexto [28].

PagedAttention e a Otimização de Memória. O PagedAttention [30] aborda especificamente o problema de gerenciamento de memória durante a inferência de LLMs, propondo uma técnica que segmenta o *cache* de atenção (*KV cache*) em “páginas”, analogamente a sistemas de paginação em memória virtual. Durante a geração autorregressiva, o modelo precisa armazenar os vetores *key* e *value* de todos os *tokens* previamente gerados, e em sequências longas esse *cache* consome memória significativa. O PagedAttention atua sobre esse *cache* de três formas, descritas a seguir.

- **Segmentação do *cache*.** Divide o *KV cache* em blocos, ou páginas, de tamanho fixo.
- **Alocação dinâmica.** As páginas são alocadas sob demanda conforme a geração avança, reduzindo o desperdício de memória.
- **Compartilhamento.** Múltiplas sequências geradas em paralelo, como em *beam search* ou inferência em lote, podem compartilhar páginas comuns, economizando memória.

Entre os benefícios estão a melhor escalabilidade, uma vez que permite processar sequências maiores ou mais sequências em paralelo com o mesmo hardware, a redução da sobrecarga de memória causada por alocação inflexível, e a preservação da qualidade das representações apesar da fragmentação do *cache* [30].

Knowledge Graph Extraction. A abordagem de extração de grafos de conhecimento propõe a transformação de textos longos e não estruturados em representações estruturadas baseadas em grafos, nas quais nós representam entidades ou conceitos e arestas representam relações entre eles [29]. O processo de criação divide-se em três etapas principais, descritas a seguir.

- **Extração de informações relevantes.** Identificação de entidades, como pessoas, organizações, locais e conceitos, e de relações causais, temporais ou hierárquicas presentes no texto.
- **Definição de conceitos e entidades.** Estruturação semântica que organiza as entidades extraídas em uma ontologia ou taxonomia coerente.
- **Canonização.** Padronização dos dados extraídos para resolver ambiguidades, como entidades distintas com nomes similares ou a mesma entidade referenciada de formas diferentes.

Essa metodologia permite a compactação de informações em estruturas ordenadas, reduzindo o espaço de armazenamento necessário enquanto mantém relações semânticas explícitas e navegáveis. Em vez de manter o texto completo em memória, o sistema armazena apenas o grafo de conhecimento e recupera subgrafos relevantes quando necessário [29].

Sparse GraphRAG e Mixture-of-PageRanks. Alonso e Millidge [29] apresentam uma abordagem denominada *Mixture-of-PageRanks*, que substitui a dependência de longos contextos em LLMs por um mecanismo de grafos esparsos em tempo real, inspirado no algoritmo PageRank, utilizado originalmente para ranqueamento de páginas web. Em vez de manter toda a informação na janela de contexto, o sistema opera de três formas, descritas a seguir.

- **Construção de grafos de entidades e relações.** Representação estruturada do conhecimento contido em documentos longos ou coleções de documentos, em que a estrutura do grafo captura dependências e relevância relativa.
- **Cálculo de relevância dinâmica.** Uso de variantes do algoritmo PageRank para identificar os nós mais importantes no contexto da consulta atual, ponderando a importância global, medida pela centralidade no grafo, e a relevância local, medida pela proximidade semântica à consulta.
- **Recuperação dinâmica.** Injeção na janela de contexto apenas das partes do grafo, ou subgrafos, relevantes à consulta, com suas conexões explícitas preservadas.

A abordagem denomina-se *Mixture-of-PageRanks* porque combina múltiplas funções de ranqueamento aplicadas simultaneamente ao mesmo grafo, capturando diferentes dimensões de relevância, como a temporal, a causal e a topológica. Entre os benefícios estão a escalabilidade, uma vez que pode processar bases de conhecimento de tamanho arbitrário, limitada apenas pela capacidade de armazenamento do grafo, a eficiência na redução do número de *tokens* necessários e a rastreabilidade, dado que grafos explícitos tornam as fontes de informação transparentes e auditáveis, facilitando a verificação e mitigando alucinações [29].

2.2.5 Síntese e Taxonomia de Técnicas Não Invasivas

A Tabela 2.2 oferece uma visão consolidada das técnicas revisadas, organizadas por categoria operacional, com indicação de seus mecanismos principais e aplicações ideais.

2.2.6 Avaliação de Eficácia, Métricas e Resultados

A eficácia das técnicas de extensão de contexto é avaliada segundo múltiplas dimensões complementares, conforme documentado na literatura revisada [31, 50, 51]. Técnicas de compressão relatam capacidade significativa de redução de *tokens* mantendo a qualidade em tarefas de *question answering* e sumarização [14, 22]. Ganhos em eficiência computacional, como menor tempo de *prefill* e menor utilização de memória, também são documentados em múltiplos trabalhos [32]. No que diz respeito ao desempenho em tarefas específicas, avaliações reportam melhorias em tarefas de *question answering*, extração de informações e sumarização, particularmente quando a informação relevante está distribuída em múltiplos documentos [31, 50]. Estudos empíricos dedicados à compressão de *prompt* examinam efeitos menos evidentes, como o impacto sobre o tamanho da resposta, a ocorrência de alucinações e a generalização entre tarefas [52], e análises voltadas à preservação de informação avaliam o quanto o conteúdo original é retido sob diferentes esquemas de compressão [53]. Esses trabalhos sugerem que ganhos de eficiência devem ser interpretados em conjunto com a fidelidade da informação preservada, e não isoladamente.

2.2.7 Benchmarks Especializados para Contexto Longo

A avaliação rigorosa de capacidades de longo contexto requer *benchmarks* que testem não apenas a capacidade técnica de processar sequências longas, mas o uso *efetivo* da informação distribuída ao longo do contexto [50, 51]. Os principais *benchmarks* utilizados são descritos a seguir.

Tabela 2.2: Taxonomia de técnicas não invasivas para extensão de contexto em LLMs

Categoria	Técnicas	Mecanismo Principal	Aplicação Ideal
Compressão de <i>Prompt</i>	<i>Hard prompt</i> (LLMLingua, Selective Context, CPC), <i>soft prompt</i> (Gist, ICAE, LLoCO), compressão para RAG e linguagem natural	Seleção de <i>tokens</i> ou sentenças, ou substituição por representações aprendidas, com preservação semântica	Respostas rápidas, redução de custo, APIs com limitação de <i>tokens</i>
Segmentação de Contexto	Janela Deslizante, Janelas Paralelas	Divisão em trechos, processamento paralelo ou sequencial e agregação	Textos multi-documento, processamento paralelo
Métodos MapReduce	Reduce Simples, QA Rank, Refine	Fragmentação (Map), processamento e agregação hierárquica (Reduce)	Síntese e análise de textos extensos, sumarização multi-documento
Memória Externa	MemGPT, In-fLLM, RIG, PagedAttention, Knowledge Graphs, Sparse GraphRAG	Armazenamento persistente e recuperação dinâmica baseada em relevância	Diálogos prolongados, bases de conhecimento extensas, análises detalhadas

- **LongBench** [50]. *Benchmark* bilíngue e multitarefa para compreensão de contexto longo, incluindo tarefas de *question answering*, sumarização e análise de código. Avalia capacidade de recuperação, síntese e raciocínio sobre informações dispersas.
- **RULER** [51]. *Framework* que mede o “tamanho real de contexto” de LLMs por meio de testes de recuperação de informação inserida em posições variadas ao longo da sequência, expondo quantitativamente fenômenos como *lost in the middle* e degradação posicional.
- **Needle-in-a-Haystack**. Teste simples mas revelador no qual um fato específico (“agulha”) é inserido em posição aleatória em um contexto longo (“palheiro”), e o modelo deve recuperá-lo com precisão. Expõe vieses posicionais, degradação de atenção e limitações práticas de janelas nominalmente grandes.

Esses *benchmarks* revelam sistematicamente que janelas de contexto nominalmente grandes frequentemente não são utilizadas de forma eficaz pelo modelo, com desempenho degradando de maneira significativa quando a informação crítica está além dos primeiros 32 a 64 mil *tokens* ou posicionada em regiões centrais da sequência [13, 50, 51].

2.2.8 Síntese e Lacunas Identificadas

Os trabalhos revisados demonstram ganhos significativos em eficiência e desempenho utilizando apenas técnicas não invasivas. Contudo, parte dos métodos de teste é inconsistente ou demasiado básica para avaliar o comportamento dos modelos em cenários reais, e as técnicas são, em geral, aplicadas de forma uniforme sobre todo o *prompt*. Essa uniformidade é uma lacuna relevante. Como *prompts* têm estrutura interna heterogênea, uma taxa de compressão única tende a comprometer regiões de alto valor informacional ou a desperdiçar orçamento em regiões redundantes. Com isso, no capítulo seguinte, este trabalho realiza a avaliação empírica das estratégias mais promissoras e formaliza um *framework* que aloca a compressão de forma seletiva entre as regiões do *prompt*.

Capítulo 3

Metodologia e *Framework* Proposto

Este capítulo é organizado em duas partes. A primeira, na Seção 3.1, apresenta a avaliação empírica e a evolução de seu protocolo. O estudo compreende uma fase preliminar, executada por meio de APIs externas, e uma fase local consolidada. A fase local, por sua vez, foi conduzida em duas etapas sequenciais: inicialmente, cinco estratégias não invasivas de tratamento de contexto foram comparadas em cinco modelos e dois orçamentos, formando a matriz local inicial e fornecendo a referência para a escolha da estratégia base; posteriormente, LLMLingua-GPT2, LLMLingua-2, Selective Context e CPC-MiniLM foram incorporados como quatro técnicas adicionais, ampliando o conjunto comparativo para nove técnicas nas dez combinações de modelo e orçamento. A segunda parte, na Seção 3.2, formaliza a arquitetura do *framework* proposto, que descreve a compressão seletiva de contexto como um problema da mochila de múltipla escolha.

3.1 Validação Empírica das Estratégias Não Invasivas

3.1.1 Objetivo e Desenho Experimental

A validação empírica tem por objetivo medir quanto de informação cada técnica de tratamento de contexto preserva, isto é, em que medida o modelo ainda recupera a resposta correta após a redução ou a segmentação da entrada, em comparação com o *baseline* sem tratamento. A questão central não é qual modelo apresenta melhor desempenho absoluto, e sim como o compromisso entre qualidade e custo varia entre técnicas, famílias de modelos e orçamentos de contexto. Para responder a essa questão sem alterar retrospectivamente o protocolo já executado, a matriz

local foi construída de forma incremental.

Na etapa inicial da matriz local, foram comparadas cinco estratégias não invasivas de tratamento de contexto, descritas a seguir.

- **Raw.** Estratégia de referência (*baseline*), em que o contexto completo é enviado ao modelo sem qualquer tratamento.
- **Parallel Window.** Segmentação do contexto em janelas processadas em paralelo, com agregação posterior das respostas parciais [49].
- **Sliding Window.** Segmentação com janelas deslizantes e sobreposição, preservando continuidade semântica entre segmentos sucessivos.
- **RIG.** Seleção de trechos por ganho de informação relevante, com ordenação do tipo *Dartboard*, priorizando o conteúdo que mais reduz a incerteza sobre a resposta [28].
- **Semantic Compression.** Compressão semântica orientada por instruções de *prompt*, sem modelo de compressão adicional [22].

Essa etapa inicial foi executada integralmente nos cinco modelos e nos dois orçamentos definidos a seguir. Após sua conclusão, quatro compressores foram acrescentados como técnicas adicionais: LLMingua-GPT2, empregado como baseline externo de compressão, e LLMingua-2, Selective Context e CPC-MiniLM, avaliados como candidatos ao conjunto de opções do MCKP. A ampliação preservou os mesmos modelos, orçamentos, *benchmarks* e critérios de agregação, permitindo apresentar as nove técnicas nas mesmas tabelas. Os quatro compressores foram executados nas dez combinações de modelo e orçamento. Uma chamada do CPC-MiniLM ao DeepSeek-R1 32B, no orçamento de 8192 *tokens*, terminou por *timeout*; essa observação é excluída das agregações dependentes da resposta e identificada nas tabelas. As implementações e adaptações dos quatro compressores são detalhadas na Seção 3.2.10.

Os modelos foram selecionados para cobrir eixos distintos de projeto, e não por serem os de maior porte. A Tabela 3.1 apresenta os cinco modelos e seu papel na matriz de avaliação, contemplando arquitetura densa em escala pequena e grande, arquitetura de mistura de especialistas esparsa e modelos ajustados para raciocínio. A escolha por famílias distintas, que reúnem Meta, Google, Alibaba, DeepSeek e OpenAI, busca reduzir a dependência do resultado em relação a um único estilo de treino.

A avaliação de cada modelo considera dois orçamentos de contexto, de 8192 e de 32768 *tokens*, correspondentes aos extremos de pressão de compressão. O orçamento de 8192 *tokens* impõe forte truncamento, exigindo que a estratégia selecione bem o

Tabela 3.1: Modelos avaliados e diversidade arquitetural da matriz de *benchmarks*

Modelo	Família	Arquitetura	Papel na matriz
llama3.1:8b	Llama (Meta)	3.1 Densa, 8B	<i>Baseline</i> pequeno e rápido
gemma4:26b	Gemma 4 (Google)	Densa, 26B	Densa de grande porte, família distinta de Llama
qwen3:30b-a3b	Qwen 3 (Alibaba)	MoE esparsa, 30B com 3B ativos	Mistura de especialistas, ativa apenas parte dos pesos
deepseek-r1:32b	DeepSeek-R1	Raciocínio, 32B	Ajustado para cadeia de raciocínio
gpt-oss:20b	GPT-OSS (OpenAI)	MoE, 20B	Família aberta da OpenAI, estilo de treino distinto

conteúdo mantido, enquanto o orçamento de 32768 *tokens* mede o teto de qualidade, uma vez que quase todo o conteúdo é acomodado. Uma calibração ampla prévia, conduzida com o modelo *baseline*, indicou que os contextos de 16384 e de 32768 *tokens* saturam em um mesmo patamar de acurácia, próximo de 0,954, de modo que o ponto intermediário não acrescenta informação relevante à curva de compromisso e foi omitido para os demais modelos. O modelo *baseline* preserva os três contextos como curva completa de referência.

A suíte de *benchmarks* reúne sete tarefas de contexto longo restritas a texto, que são LongBench, ZeroSCROLLS, Natural Questions, TriviaQA, HotpotQA, MuSiQue e sumarização de reuniões. *Benchmarks* puramente sintéticos, como o Needle-in-a-Haystack e o RULER, foram deixados fora da matriz principal por não integrarem a suíte de referência adotada. Para cada combinação de modelo, orçamento de contexto, estratégia e *benchmark*, registram-se a acurácia, a latência e a taxa de compressão efetiva. Valores não disponíveis são indicados por *n/d*.

As decisões metodológicas relativas à seleção dos modelos, aos orçamentos de contexto e à composição da suíte estão registradas em `tests/README.md` e em `tests/ollama-local/README_DECISAO_TESTES.md`. Nos dois orçamentos comparáveis, a etapa inicial da matriz local produziu 1100 resultados individuais: 22 casos para cada uma das 50 combinações de modelo, orçamento e estratégia. Esses resultados correspondem a 350 células agregadas por *benchmark*. Os artefatos de execução incluem os *logs*, os resultados por caso nos formatos CSV e JSON e as tabelas agregadas disponíveis em `tests/ollama-local/benchmark_matrix_artigo`. A execução adicional do Llama 3.1 8B com 16384 *tokens* foi preservada como ponto intermediário de calibração, mas não integra as tabelas comparativas dos dois orçamentos

extremos.

3.1.2 Evolução do Protocolo Experimental

O desenho experimental consolidado na Seção 3.1.1 resulta de uma evolução do protocolo de avaliação conduzida em duas fases. A fase preliminar utilizou modelos servidos por interfaces de programação externas, especificamente a Groq e a Cerebras, o que permitiu avaliar modelos de grande porte, como o Llama 3.3 70B e o GPT-OSS 120B, sobre um conjunto inicial de *benchmarks* composto por Needle-in-a-Haystack, RULER e LongBench. Essa configuração aproveitou o acesso gratuito então oferecido por esses provedores.

Ao longo do trabalho, contudo, as condições de acesso foram alteradas. Os planos gratuitos da Groq e da Cerebras foram revisados, diversos modelos foram descontinuados ou indisponibilizados, e os limites de requisição passaram a restringir a execução da matriz completa. Essa alteração faz parte do processo de pesquisa apoiado em infraestrutura de terceiros, cuja disponibilidade não é controlada pelo autor. Para preservar a reprodutibilidade e a autonomia da avaliação, a segunda fase migrou para modelos executados localmente por meio do Ollama, o que motivou a matriz de cinco modelos de arquiteturas distintas nos orçamentos de 8192 e de 32768 *tokens* descrita na Seção 3.1.1. Dentro dessa fase local, as cinco estratégias originais foram avaliadas primeiro; os quatro compressores adicionais foram incorporados posteriormente, sem alterar os modelos, os orçamentos, a suíte ou os critérios de agregação.

3.1.3 Fase Preliminar, Avaliação via APIs Externas

Os resultados da fase preliminar são reportados a seguir e preservados por transparência do processo. Essa fase avaliou quatro modelos, que são o Llama 3.1 8B, o Llama 3.3 70B, o GPT-OSS 20B e o GPT-OSS 120B, sobre os *benchmarks* Needle-in-a-Haystack, RULER e LongBench, com a acurácia e a latência média de resposta registradas por combinação de modelo, estratégia e *benchmark*. A coluna **Média** corresponde à média das métricas nos *benchmarks* avaliados. Valores não disponíveis, decorrentes sobretudo dos limites de requisição descritos, são indicados por *n/d*.

3.1.4 Resultados de Acurácia da Fase Preliminar

A Tabela 3.2 apresenta a acurácia obtida por cada estratégia nos três *benchmarks*, agrupada por modelo. Em todas as tabelas de resultados deste capítulo, valores em **negrito** destacam o melhor resultado por coluna dentro de cada modelo e valores

sublinhados o segundo melhor; nas tabelas de latência, menor é melhor. A marcação n/d indica ausência de dados na configuração correspondente.

Tabela 3.2: Acurácia por estratégia, modelo e *benchmark* de contexto longo

	Benchmarks de Contexto Longo			
Estratégia	Needle	RULER	LongBench	Média
<i>Llama 3.1 8B</i>				
<i>Parallel Window</i>	1.00	1.00	0.94	0.98
<i>Raw</i>	1.00	<u>0.94</u>	0.94	<u>0.96</u>
RIG	1.00	1.00	0.94	0.98
<i>Semantic Compression</i>	<u>0.33</u>	0.56	0.94	0.61
<i>Sliding Window</i>	1.00	1.00	0.94	0.98
<i>Llama 3.3 70B</i>				
<i>Parallel Window</i>	1.00	1.00	<u>0.94</u>	<u>0.98</u>
<i>Raw</i>	1.00	1.00	1.00	1.00
RIG	1.00	1.00	<u>0.94</u>	<u>0.98</u>
<i>Semantic Compression</i>	<u>0.33</u>	<u>0.69</u>	0.38	0.47
<i>Sliding Window</i>	1.00	1.00	<u>0.94</u>	<u>0.98</u>
<i>GPT-OSS 20B</i>				
<i>Parallel Window</i>	1.00	0.69	<i>n/d</i>	0.85
<i>Raw</i>	1.00	0.78	0.50	0.76
RIG	1.00	0.50	<i>n/d</i>	0.75
<i>Semantic Compression</i>	<u>0.73</u>	0.00	<i>n/d</i>	0.37
<i>Sliding Window</i>	1.00	<u>0.72</u>	<i>n/d</i>	0.86
<i>GPT-OSS 120B</i>				
<i>Parallel Window</i>	1.00	<u>0.58</u>	<u>0.00</u>	0.53
<i>Raw</i>	1.00	0.56	0.17	<u>0.58</u>
RIG	<u>0.97</u>	0.44	<i>n/d</i>	0.71
<i>Semantic Compression</i>	1.00	0.06	<i>n/d</i>	0.53
<i>Sliding Window</i>	0.87	0.64	<u>0.00</u>	0.50

Os resultados de acurácia indicam que as estratégias de segmentação, tanto *Parallel Window* quanto *Sliding Window*, e a seleção por RIG mantêm desempenho elevado nos modelos Llama, com média próxima de 0.98. A estratégia *Raw* apresenta o melhor resultado isolado no Llama 3.3 70B, com média de 1.00, o que sugere que modelos maiores conseguem aproveitar o contexto completo de forma mais eficaz quando não há restrição de orçamento. A *Semantic Compression* apresenta o pior desempenho de acurácia entre as estratégias avaliadas, com quedas acentuadas no *benchmark* RULER, o que é compatível com a perda de informação associada a uma compressão agressiva e uniforme. Nos modelos GPT-OSS, observa-se degradação geral de acurácia no RULER e no LongBench, com maior dispersão entre as estratégias.

3.1.5 Resultados de Latência da Fase Preliminar

A Tabela 3.3 apresenta a latência média de resposta, em milissegundos, por estratégia, modelo e *benchmark*.

Tabela 3.3: Latência média de resposta (ms) por estratégia, modelo e *benchmark*

	Benchmarks de Contexto Longo			
Estratégia	Needle	RULER	LongBench	Média
<i>Llama 3.1 8B</i>				
<i>Parallel Window</i>	6200	<u>3700</u>	2400	4100
<i>Raw</i>	318	1000	<u>344</u>	554
RIG	6300	3900	6300	5500
<i>Semantic Compression</i>	11700	7300	11700	10200
<i>Sliding Window</i>	<u>2300</u>	3800	286	<u>2129</u>
<i>Llama 3.3 70B</i>				
<i>Parallel Window</i>	271	2300	346	972
<i>Raw</i>	247	280	316	281
RIG	3200	2500	1500	2400
<i>Semantic Compression</i>	3600	5100	965	3222
<i>Sliding Window</i>	<u>262</u>	<u>517</u>	<u>336</u>	<u>372</u>
<i>GPT-OSS 20B</i>				
<i>Parallel Window</i>	7134	3489	<i>n/d</i>	5312
<i>Raw</i>	2775	<u>298</u>	2766	1946
RIG	<u>7055</u>	2766	<i>n/d</i>	<u>4911</u>
<i>Semantic Compression</i>	14676	130	<i>n/d</i>	7403
<i>Sliding Window</i>	7527	3399	<i>n/d</i>	5463
<i>GPT-OSS 120B</i>				
<i>Parallel Window</i>	<u>7165</u>	3372	159	<u>3565</u>
<i>Raw</i>	3507	<u>2141</u>	231	1960
RIG	7272	3085	<i>n/d</i>	5179
<i>Semantic Compression</i>	16670	2071	<i>n/d</i>	9371
<i>Sliding Window</i>	7681	3638	71	3797

Os resultados de latência mostram que a estratégia *Raw* é consistentemente a mais rápida, o que é esperado, pois não há custo adicional de segmentação, seleção ou compressão. Entre as estratégias que efetivamente tratam o contexto, a *Sliding Window* apresenta a menor latência no Llama 3.1 8B, com média de aproximadamente 2,1 segundos, enquanto RIG e *Semantic Compression* apresentam as maiores latências, associadas ao custo de seleção por ganho de informação e ao custo de compressão semântica. Nos modelos GPT-OSS, as latências são substancialmente maiores para todas as estratégias baseadas em segmentação e compressão, com a *Semantic Compression* atingindo os maiores tempos observados.

3.1.6 Discussão da Fase Preliminar

A análise conjunta das Tabelas 3.2 e 3.3 evidencia um compromisso entre acurácia e latência. A estratégia *Raw* oferece a menor latência, porém sem qualquer mecanismo de tratamento de contexto, o que limita sua aplicabilidade em cenários que excedem a janela nominal do modelo. A *Semantic Compression*, por outro lado, combina baixa acurácia com alta latência nos cenários avaliados, o que a torna pouco atrativa como estratégia isolada e uniforme. Entre as estratégias que tratam o contexto, a *Sliding Window* apresenta o melhor equilíbrio na fase preliminar, mantendo acurácia média de 0.98 no Llama 3.1 8B, equivalente à das demais estratégias de segmentação e seleção, com a menor latência entre elas.

Um ponto que atravessa os dois conjuntos de resultados é que nenhuma estratégia uniforme domina todas as demais em todos os modelos e *benchmarks*. Esse comportamento é consistente com a hipótese central desta dissertação, segundo a qual o tratamento uniforme do *prompt* desperdiça orçamento em regiões redundantes e compromete regiões de alto valor informacional. A fase preliminar também expôs uma limitação do conjunto de *benchmarks* então utilizado, discutida a seguir, que motivou a revisão da suíte de avaliação adotada na fase local.

3.1.7 Revisão da Suíte de Benchmarks

A fase preliminar apoiou-se em dois *benchmarks* de natureza sintética, o Needle-in-a-Haystack e o RULER. O Needle-in-a-Haystack insere um fato isolado, a agulha, em um volume extenso de texto de preenchimento e verifica se o modelo consegue recuperá-lo, variando a posição da agulha e o comprimento do contexto [54]. O RULER estende essa ideia com múltiplas agulhas, rastreamento de variáveis e tarefas de agregação, mas mantém a construção sintética e controlada [51]. Esses *benchmarks* são úteis para medir a capacidade de recuperação posicional, porém avaliam uma forma superficial de compreensão de contexto longo, na qual a resposta depende de localizar um trecho específico, e não de integrar informação distribuída ao longo da entrada. Pontuações próximas do máximo no Needle-in-a-Haystack, portanto, não se traduzem necessariamente em bom desempenho em tarefas reais.

Como o objetivo desta dissertação é medir quanto de informação uma estratégia de compressão efetivamente preserva, a fase local adota uma suíte de *benchmarks* de tarefas realistas, mais discriminante para esse propósito. O LongBench [50] e o ZeroSCROLLS [55] reúnem múltiplas tarefas de contexto longo em formato de avaliação sem ajuste específico, cobrindo compreensão, sumarização e resposta a perguntas. A resposta a perguntas de múltiplos saltos é avaliada pelo HotpotQA [56] e pelo MuSiQue [57], que exigem combinar evidências dispersas em vários documentos e, por isso, penalizam compressões que descartam fatos de ligação. A

resposta a perguntas de domínio aberto é avaliada pelo Natural Questions [58] e pelo TriviaQA [59], que utilizam consultas reais de usuários e questões sobre documentos. A sumarização orientada por consulta é avaliada pelo QMSum [60], que requer selecionar e resumir trechos relevantes ao longo de diálogos extensos.

A vantagem dessa suíte é que as tarefas dependem de agregação, de raciocínio de múltiplos saltos e da preservação fiel de informação distribuída, que são exatamente as propriedades que a alocação de compressão por partição precisa proteger. Uma estratégia pode obter pontuação alta no Needle-in-a-Haystack ao manter um único trecho, mas falhar em resposta a perguntas de múltiplos saltos se descartar um fato de ligação. Por essa razão, a suíte adotada na fase local mede de forma mais fiel o compromisso entre redução de *tokens* e informação preservada, que é o critério central para a avaliação do *framework* proposto.

3.1.8 Resultados da Matriz Local Ampliada

A matriz local abrange cinco modelos, o Llama 3.1 8B, o Gemma 4 26B, o Qwen 3 30B-A3B, o DeepSeek-R1 32B e o GPT-OSS 20B, e foi construída nas duas etapas descritas na Seção 3.1.1. A etapa inicial avaliou *Raw*, *Sliding Window*, *Parallel Window*, *Semantic Compression* e RIG em todas as combinações de modelo e orçamento. Posteriormente, LLMLingua-GPT2, LLMLingua-2, Selective Context e CPC-MiniLM foram incorporados como técnicas adicionais, elevando para nove o número de linhas comparáveis em cada bloco da matriz. Essa ampliação foi executada nos cinco modelos e nos dois orçamentos.

A Tabela 3.4 apresenta a acurácia por técnica e *benchmark* para o orçamento de 8192 *tokens*, e a Tabela 3.5 apresenta os mesmos resultados para o orçamento de 32768 *tokens*. Em ambas, cada técnica ocupa uma linha dentro do respectivo bloco de modelo, e a coluna Média corresponde à média macro das pontuações nos sete *benchmarks* da suíte revisada. Na combinação CPC-MiniLM, DeepSeek-R1 32B e 8192 *tokens*, a pontuação do QMSum e a média macro são calculadas sobre as duas respostas válidas, em vez das três previstas, e identificadas por um asterisco.

Tabela 3.4: Acurácia por técnica e *benchmark* nos cinco modelos, orçamento de 8192 *tokens*. As colunas abreviam LongBench (LB), ZeroSCROLLS (ZS), Natural Questions (NQ), TriviaQA (TQA), HotpotQA (HQA), MuSiQue (MSQ) e QMSum (QMS). O asterisco identifica a combinação com uma resposta excluída por *timeout*

	Suítes		Open-domain QA		Multi-hop QA		Sum.	
Técnica	LB	ZS	NQ	TQA	HQA	MSQ	QMS	Média
Llama 3.1 8B								
Raw	1.000	0.443	<u>0.362</u>	1.000	0.667	0.417	0.273	0.594
Sliding Window	1.000	0.642	<u>0.362</u>	1.000	0.667	<u>0.333</u>	0.379	0.626
Parallel Window	1.000	0.329	<u>0.362</u>	1.000	0.667	0.417	0.240	0.573
Semantic Compression	<u>0.938</u>	0.377	0.691	1.000	0.667	<u>0.333</u>	0.239	<u>0.606</u>
RIG	1.000	0.404	<u>0.362</u>	1.000	0.667	0.417	0.083	0.562
LLMLingua-GPT2	0.688	0.273	0.050	1.000	<u>0.333</u>	0.083	0.222	0.379
LLMLingua-2	0.812	<u>0.619</u>	0.080	1.000	<u>0.333</u>	<u>0.333</u>	0.262	0.492
Selective Context	0.688	0.572	<u>0.362</u>	1.000	0.000	0.417	0.321	0.480
CPC-MiniLM	0.688	0.583	<u>0.362</u>	1.000	<u>0.333</u>	0.417	<u>0.335</u>	0.531
Gemma 4 26B								
Raw	0.938	<u>0.333</u>	0.671	1.000	0.667	0.333	0.000	<u>0.563</u>
Sliding Window	0.938	<u>0.333</u>	0.671	1.000	0.667	0.333	0.068	0.573
Parallel Window	0.750	<u>0.333</u>	0.671	1.000	0.667	0.333	0.014	0.538
Semantic Compression	<u>0.812</u>	<u>0.333</u>	0.000	1.000	<u>0.333</u>	0.333	0.061	0.411
RIG	0.500	<u>0.333</u>	0.671	1.000	0.000	0.333	0.088	0.418
LLMLingua-GPT2	0.250	0.000	0.337	1.000	<u>0.333</u>	0.000	0.000	0.274
LLMLingua-2	0.938	0.583	<u>0.341</u>	1.000	<u>0.333</u>	0.000	0.007	0.457
Selective Context	0.688	<u>0.333</u>	0.337	0.667	0.000	0.333	<u>0.116</u>	0.353
CPC-MiniLM	0.688	<u>0.333</u>	0.671	1.000	0.667	0.333	0.138	0.547
Qwen 3 30B-A3B								
Raw	0.750	0.389	<u>0.667</u>	1.000	0.333	0.333	0.141	<u>0.516</u>
Sliding Window	0.750	0.222	<u>0.667</u>	1.000	<u>0.000</u>	0.333	0.000	0.425
Parallel Window	0.750	0.222	1.000	1.000	<u>0.000</u>	0.333	0.010	0.474
Semantic Compression	<u>0.625</u>	0.417	0.333	1.000	<u>0.000</u>	<u>0.000</u>	<u>0.153</u>	0.361
RIG	0.750	0.409	1.000	1.000	<u>0.000</u>	0.333	0.157	0.521
LLMLingua-GPT2	0.000	0.111	<u>0.667</u>	0.333	0.333	<u>0.000</u>	0.000	0.206
LLMLingua-2	0.750	0.592	<u>0.667</u>	1.000	<u>0.000</u>	<u>0.000</u>	0.000	0.430
Selective Context	0.500	0.222	0.333	0.333	<u>0.000</u>	0.333	0.083	0.258
CPC-MiniLM	0.500	<u>0.583</u>	<u>0.667</u>	0.667	<u>0.000</u>	0.333	0.005	0.394

Continua na próxima página

Tabela 3.4 – continuação

	Suítes		Open-domain QA		Multi-hop QA		Sum.	
Técnica	LB	ZS	NQ	TQA	HQA	MSQ	QMS	Média
DeepSeek-R1 32B								
Raw	0.750	0.620	0.164	1.000	1.000	0.833	0.344	0.673
Sliding Window	0.750	0.422	0.142	1.000	1.000	0.500	0.335	0.593
Parallel Window	0.500	0.500	0.142	1.000	0.667	0.583	0.230	0.517
Semantic Compression	0.750	0.629	0.172	1.000	0.667	0.333	0.213	0.538
RIG	0.750	0.520	0.142	1.000	0.667	0.667	0.380	0.589
LLMLingua-GPT2	0.500	0.306	0.142	0.667	0.333	0.000	0.302	0.321
LLMLingua-2	0.500	0.650	0.150	1.000	1.000	0.333	0.432	0.581
Selective Context	0.500	0.690	0.209	0.667	1.000	0.667	0.302	0.576
CPC-MiniLM*	0.500	0.743	0.172	1.000	1.000	0.583	0.291	0.613
GPT-OSS 20B								
Raw	0.688	0.333	0.050	1.000	0.667	0.917	0.052	0.530
Sliding Window	0.938	0.546	0.050	1.000	1.000	0.417	0.250	0.600
Parallel Window	0.688	0.523	0.050	1.000	0.667	0.333	0.135	0.485
Semantic Compression	0.750	0.476	0.006	1.000	0.500	0.000	0.339	0.439
RIG	0.938	0.298	0.050	1.000	0.833	0.667	0.313	0.586
LLMLingua-GPT2	0.438	0.000	0.114	1.000	0.333	0.000	0.144	0.290
LLMLingua-2	0.938	0.467	0.025	1.000	0.333	0.333	0.313	0.487
Selective Context	0.312	0.391	0.028	1.000	0.667	0.333	0.255	0.427
CPC-MiniLM	0.688	0.707	0.050	1.000	0.667	0.667	0.315	0.585

Tabela 3.5: Acurácia por técnica e *benchmark* nos cinco modelos, orçamento de 32768 *tokens*. As abreviações de coluna seguem a Tabela 3.4

Técnica	Suítes		Open-domain QA		Multi-hop QA		Sum.	Média
	LB	ZS	NQ	TQA	HQA	MSQ	QMS	
<i>Llama 3.1 8B</i>								
<i>Raw</i>	1.000	0.619	<u>0.362</u>	1.000	0.667	0.417	0.326	0.627
<i>Sliding Window</i>	1.000	<u>0.642</u>	<u>0.362</u>	1.000	0.667	<u>0.333</u>	0.379	<u>0.626</u>
<i>Parallel Window</i>	1.000	0.329	<u>0.362</u>	1.000	0.667	0.417	0.247	0.574
<i>Semantic Compression</i>	<u>0.938</u>	0.354	0.691	1.000	0.667	<u>0.333</u>	0.158	0.592
RIG	1.000	0.404	<u>0.362</u>	1.000	0.667	0.417	0.083	0.562
LLMLingua-GPT2	0.688	0.190	0.050	1.000	<u>0.333</u>	0.083	0.222	0.367
LLMLingua-2	0.812	0.606	0.080	1.000	<u>0.333</u>	<u>0.333</u>	<u>0.356</u>	0.503
Selective Context	0.688	0.656	<u>0.362</u>	1.000	0.000	0.417	0.290	0.487
CPC-MiniLM	0.688	0.583	<u>0.362</u>	1.000	<u>0.333</u>	0.417	0.335	0.531

Continua na próxima página

Tabela 3.5 – continuação

	Suítes		Open-domain QA		Multi-hop QA		Sum.	
Técnica	LB	ZS	NQ	TQA	HQA	MSQ	QMS	Média
Gemma 4 26B								
Raw	0.938	0.583	0.671	1.000	0.667	0.000	0.129	0.570
Sliding Window	0.938	0.373	0.671	1.000	0.333	0.333	0.000	0.521
Parallel Window	0.750	0.333	0.339	1.000	1.000	0.333	0.014	0.538
Semantic Compression	0.812	0.583	0.000	1.000	0.333	0.333	0.109	0.453
RIG	0.750	0.333	0.671	1.000	0.333	0.333	0.054	0.496
LLMLingua-GPT2	0.375	0.000	0.337	1.000	0.333	0.000	0.000	0.292
LLMLingua-2	0.938	0.333	0.333	1.000	0.667	0.000	0.000	0.467
Selective Context	0.688	0.222	0.337	0.667	0.000	0.333	0.102	0.336
CPC-MiniLM	0.688	0.583	0.671	1.000	0.667	0.333	0.068	0.573
Qwen 3 30B-A3B								
Raw	0.750	0.472	0.667	1.000	0.333	0.333	0.000	0.508
Sliding Window	0.750	0.222	0.667	1.000	0.000	0.333	0.000	0.425
Parallel Window	0.750	0.222	1.000	1.000	0.000	0.333	0.010	0.474
Semantic Compression	0.625	0.306	0.333	1.000	0.000	0.000	0.112	0.339
RIG	0.750	0.409	1.000	1.000	0.000	0.333	0.157	0.521
LLMLingua-GPT2	0.000	0.000	0.667	0.333	0.333	0.000	0.000	0.190
LLMLingua-2	0.750	0.472	0.667	1.000	0.000	0.000	0.000	0.413
Selective Context	0.500	0.472	0.333	0.333	0.000	0.333	0.000	0.282
CPC-MiniLM	0.500	0.583	0.667	0.667	0.000	0.333	0.005	0.394
DeepSeek-R1 32B								
Raw	0.750	0.813	0.164	1.000	1.000	0.833	0.395	0.708
Sliding Window	0.750	0.422	0.142	1.000	1.000	0.500	0.335	0.593
Parallel Window	0.500	0.500	0.142	1.000	0.667	0.583	0.269	0.523
Semantic Compression	0.750	0.624	0.172	1.000	0.667	0.333	0.307	0.550
RIG	0.750	0.520	0.142	1.000	0.667	0.667	0.380	0.589
LLMLingua-GPT2	0.500	0.389	0.142	0.667	0.333	0.000	0.302	0.333
LLMLingua-2	0.500	0.659	0.150	1.000	1.000	0.333	0.407	0.578
Selective Context	0.500	0.694	0.209	0.667	1.000	0.667	0.355	0.585
CPC-MiniLM	0.500	0.827	0.172	1.000	1.000	0.583	0.354	0.634
GPT-OSS 20B								
Raw	0.688	0.809	0.050	1.000	0.667	0.917	0.329	0.637
Sliding Window	0.938	0.537	0.050	1.000	1.000	0.750	0.255	0.647
Parallel Window	0.688	0.510	0.050	1.000	1.000	0.667	0.217	0.590
Semantic Compression	0.750	0.580	0.004	1.000	0.667	0.333	0.141	0.496
RIG	0.938	0.298	0.050	1.000	0.667	0.667	0.204	0.546
LLMLingua-GPT2	0.438	0.000	0.114	1.000	0.333	0.000	0.144	0.290
LLMLingua-2	0.938	0.142	0.025	1.000	0.333	0.333	0.295	0.438
Selective Context	0.312	0.333	0.028	1.000	0.667	0.333	0.163	0.405
CPC-MiniLM	0.688	0.373	0.050	1.000	0.667	0.667	0.315	0.537

A Tabela 3.6 resume a latência média de resposta e a taxa de compressão efetiva por estratégia, medida como a fração de *tokens* retida após o tratamento do contexto, em que o valor 1.000 corresponde à ausência de compressão.

Tabela 3.6: Latência média (ms) e taxa de compressão efetiva por técnica nos cinco modelos. A taxa retida é a média das taxas nos dois orçamentos, em que 1.000 corresponde à ausência de compressão. O asterisco identifica a média de latência calculada após a exclusão de uma chamada encerrada por *timeout*

Técnica	Latência (ms)		Taxa retida
	8k	32k	
<i>Llama 3.1 8B</i>			
<i>Raw</i>	8228	16588	1.000
<i>Sliding Window</i>	6764	<u>6994</u>	0.857
<i>Parallel Window</i>	6922	7077	0.856
<i>Semantic Compression</i>	35572	87078	0.367
RIG	4366	4608	0.692
LLMLingua-GPT2	8451	10127	0.508
LLMLingua-2	9484	12385	0.470
Selective Context	10344	14825	0.566
CPC-MiniLM	<u>6736</u>	10345	0.524
<i>Gemma 4 26B</i>			
<i>Raw</i>	15818	19361	1.000
<i>Sliding Window</i>	14752	<u>13887</u>	0.857
<i>Parallel Window</i>	14570	14204	0.856
<i>Semantic Compression</i>	306142	354957	0.328
RIG	13247	12614	0.692
LLMLingua-GPT2	18237	18864	0.508
LLMLingua-2	17477	20048	0.470
Selective Context	19508	22037	0.566
CPC-MiniLM	<u>14307</u>	15755	0.524

Continua na próxima página

Tabela 3.6 – continuação

Técnica	Latência (ms)		Taxa retida
	8k	32k	
Qwen 3 30B-A3B			
Raw	13461	20345	1.000
Sliding Window	12327	<u>12340</u>	0.857
Parallel Window	12371	12405	0.856
Semantic Compression	43051	51182	0.417
RIG	10363	10381	0.692
LLMLingua-GPT2	14144	17128	0.508
LLMLingua-2	14503	18423	0.470
Selective Context	16140	21096	0.566
CPC-MiniLM	<u>11840</u>	15836	0.524
DeepSeek-R1 32B			
Raw	81871	111013	1.000
Sliding Window	76534	<u>79862</u>	0.857
Parallel Window	77592	82807	0.856
Semantic Compression	198572	265406	0.376
RIG	62385	63864	0.692
LLMLingua-GPT2	<u>72195</u>	80249	0.507
LLMLingua-2	78809	90642	0.469
Selective Context	108964	89828	0.566
CPC-MiniLM	75366*	83529	0.524
GPT-OSS 20B			
Raw	11233	14567	1.000
Sliding Window	<u>9551</u>	10617	0.857
Parallel Window	10150	10650	0.856
Semantic Compression	139627	231207	0.355
RIG	8615	<u>10639</u>	0.692
LLMLingua-GPT2	15449	16308	0.507
LLMLingua-2	14546	15648	0.469
Selective Context	15940	17707	0.566
CPC-MiniLM	<u>9214</u>	11079	0.524

A leitura conjunta das três tabelas mostra que nenhuma estratégia domina os cinco modelos. Sob o orçamento de 8192 *tokens*, a maior acurácia média é obtida pela *Sliding Window* no Llama 3.1 8B, no Gemma 4 26B e no GPT-OSS 20B, pela RIG no Qwen 3 30B-A3B e pela estratégia *Raw* no DeepSeek-R1 32B. Sob o orçamento de 32768 *tokens*, a estratégia *Raw* lidera no Llama 3.1 8B e no DeepSeek-R1 32B, a

RIG permanece à frente no Qwen 3 30B-A3B e a *Sliding Window* lidera no GPT-OSS 20B. No Gemma 4 26B, CPC-MiniLM alcança 0.573 e supera, por pequena margem, a estratégia *Raw*, com 0.570. Essa é a única combinação em que um dos quatro compressores acrescentados ocupa a primeira posição entre as nove técnicas. No DeepSeek-R1 32B, CPC-MiniLM ocupa a segunda posição nos dois orçamentos, com 0.613 e 0.634, mas permanece abaixo da estratégia *Raw*, com 0.673 e 0.708. No GPT-OSS 20B, CPC-MiniLM obtém 0.585 e 0.537, abaixo da *Sliding Window*, que alcança 0.600 e 0.647.

Em latência, a RIG é a estratégia mais rápida entre as que tratam o contexto em todos os cinco modelos sob o orçamento de 8192 *tokens* e em quatro dos cinco sob o orçamento de 32768 *tokens*, com a única exceção do GPT-OSS 20B nesse orçamento, em que a RIG e a *Sliding Window* praticamente empatam. Entre os quatro compressores adicionais, CPC-MiniLM apresenta a menor latência em sete das dez combinações; LLMingua-GPT2 é mais rápido no Llama 3.1 8B sob 32768 *tokens* e nos dois orçamentos do DeepSeek-R1 32B. A *Semantic Compression* apresenta os maiores tempos em todos os modelos, com valores de uma ordem de grandeza acima das demais e chegando a 354957 ms no Gemma 4 26B sob o orçamento de 32768 *tokens*, uma vez que a compressão semântica exige chamadas adicionais ao próprio modelo. Em taxa de compressão, a *Semantic Compression* é a mais agressiva, retraindo entre 0.328 e 0.417 dos *tokens* conforme o modelo. Entre os quatro compressores adicionais, LLMingua-2 retém a menor fração média, enquanto Selective Context retém a maior.

3.1.9 Seleção da Estratégia Base

Na etapa inicial da matriz local, a *Sliding Window* apresenta, na média dos dois orçamentos, a maior acurácia entre as estratégias que tratam o contexto no Llama 3.1 8B, no Gemma 4 26B, no DeepSeek-R1 32B e no GPT-OSS 20B. A RIG é superior apenas no Qwen 3 30B-A3B. Consideradas separadamente as dez combinações de modelo e orçamento, a *Sliding Window* alcança a maior acurácia dessa classe em sete, a RIG em duas e a *Parallel Window* em uma. A RIG oferece menor latência média, mas sua acurácia é inferior à da *Sliding Window* em quatro dos cinco modelos. No GPT-OSS 20B, por exemplo, a *Sliding Window* obtém 0.600 e 0.647 nos orçamentos de 8192 e 32768 *tokens*, enquanto a RIG obtém 0.586 e 0.546. Esse resultado, obtido antes da ampliação com os quatro compressores, sustenta a *Sliding Window* como estratégia base da formulação. A etapa posterior não substitui essa comparação; ela caracteriza compressores adicionais que podem compor as opções do MCKP.

A seção seguinte formaliza uma alternativa que distribui a compressão de forma seletiva entre as partições do *prompt*. Embora a *Sliding Window* seja adotada como

estratégia base, a formulação admite outras formas de segmentação como ponto de partida para o particionamento.

3.2 Framework Proposto

3.2.1 Motivação e Visão Geral

O *framework* proposto trata a compressão de *prompt* como um problema de alocação de orçamento. Em vez de aplicar um único compressor e uma única taxa a todo o *prompt*, o *framework* particiona a entrada, estima a importância de cada partição para a tarefa e escolhe, para cada partição, uma opção de compressão que maximiza a informação preservada sob uma restrição global de orçamento de *tokens*. A formulação corresponde a um problema da mochila de múltipla escolha (*Multiple-Choice Knapsack Problem*, MCKP), em que cada partição contribui com exatamente uma opção escolhida entre um conjunto de compressores candidatos.

A Figura 3.1 apresenta a arquitetura conceitual do *framework*, organizada em cinco etapas encadeadas. A figura distingue os artefatos produzidos em cada etapa, desde as partições do *prompt* até a entrada comprimida enviada ao modelo.

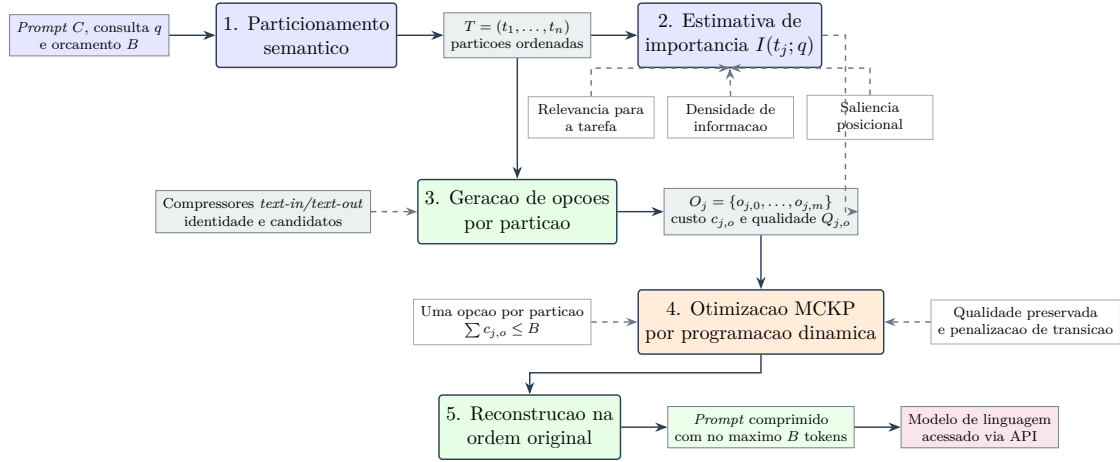


Figura 3.1: Arquitetura conceitual do *framework* proposto. O sistema transforma cada partição em alternativas com custo e qualidade, seleciona uma alternativa por partição sob o orçamento global e reconstrói o *prompt* antes da chamada ao modelo.

3.2.2 Particionamento e Orçamento

Seja C o *prompt* de entrada, com comprimento $|C|$ medido em *tokens*. O *framework* particiona C em uma sequência de n partições $T = (t_1, t_2, \dots, t_n)$, de modo que cada partição corresponda a uma unidade semântica coerente, como uma instrução, uma pergunta, um documento recuperado ou um trecho de fundo. Define-se um

orçamento efetivo de contexto $B \leq |C|$, que representa o número máximo de *tokens* que o *prompt* comprimido pode ocupar após a seleção das opções de compressão.

3.2.3 Função de Importância

Para cada partição t_j , define-se uma função de importância $I(t_j; q)$ que quantifica a relevância da partição para a consulta q . A função combina múltiplos fatores por meio de uma soma ponderada, conforme a Equação 3.1.

$$I(t_j; q) = \sum_i w_i \varphi_i(t_j, q), \quad (3.1)$$

onde cada $\varphi_i(t_j, q)$ é um componente de relevância e w_i é o peso associado. Os componentes considerados incluem a relevância da partição para a tarefa, a densidade de informação da partição e a saliência posicional, esta última motivada pelo viés *lost in the middle* discutido na Seção 1.1.1. A função de importância orienta a alocação de orçamento, atribuindo maior prioridade às partições cuja perda de conteúdo teria maior impacto sobre a resposta.

3.2.4 Opções de Compressão e Modelo de Qualidade

Para cada partição t_j , o *framework* considera um conjunto de opções de compressão $O_j = \{o_{j,0}, o_{j,1}, \dots, o_{j,m}\}$. Cada opção $o = (A_k, p)$ corresponde à aplicação de um compressor A_k com um parâmetro de operação p , como uma taxa de compressão alvo. A opção $o_{j,0}$ corresponde ao compressor identidade A_0 , que preserva a partição integralmente e serve de referência para o custo e a qualidade máximos daquela partição.

Cada opção possui um custo $c_{j,o}$, medido pelo número de *tokens* da partição após a compressão, e uma qualidade $Q_{j,o}$, que estima a informação preservada pela opção. A qualidade combina a importância da partição com a fidelidade da opção de compressão, conforme a Equação 3.2.

$$Q_{j,o} = I(t_j; q) \cdot f_{j,o}, \quad (3.2)$$

onde $f_{j,o} \in [0, 1]$ representa a fração de informação da partição preservada pela opção o . O compressor identidade satisfaz $f_{j,0} = 1$, e opções mais agressivas apresentam valores menores de $f_{j,o}$, capturando o compromisso entre redução de *tokens* e perda de conteúdo.

3.2.5 Restrições de Compatibilidade

Nem toda combinação de opções entre partições adjacentes é adequada, pois transições abruptas de estilo ou de nível de compressão entre trechos vizinhos podem prejudicar a coerência do *prompt* reconstruído. Define-se uma função de compatibilidade $M(o, o')$ entre opções de partições adjacentes, que penaliza combinações incompatíveis. A compatibilidade é incorporada como uma penalização de transição na função objetivo, por meio de um termo $\mu \sum_j d(o_j, o_{j+1})$, onde $d(\cdot, \cdot)$ mede a distância entre opções adjacentes e μ controla a intensidade da penalização.

3.2.6 Formulação como Extensão do Problema da Mochila de Múltipla Escolha

A seleção das opções de compressão é formulada como uma extensão do MCKP com penalização entre escolhas adjacentes. Para cada partição t_j e cada opção $o \in O_j$, define-se uma variável binária $x_{j,o} \in \{0, 1\}$, que assume o valor 1 quando a opção o é escolhida para a partição t_j . Denota-se por o_j a única opção cuja variável assume valor 1 na partição j . O problema consiste em maximizar a qualidade total preservada sujeita ao orçamento e à escolha de exatamente uma opção por partição, conforme as Equações 3.3 a 3.6. Quando $\mu = 0$, a formulação se reduz ao MCKP clássico; quando $\mu > 0$, o termo de transição incorpora a coerência entre partições vizinhas.

$$\max_x \quad \sum_{j=1}^n \sum_{o \in O_j} x_{j,o} Q_{j,o} - \mu \sum_{j=1}^{n-1} d(o_j, o_{j+1}) \quad (3.3)$$

$$\text{s.a.} \quad \sum_{j=1}^n \sum_{o \in O_j} x_{j,o} c_{j,o} \leq B \quad (3.4)$$

$$\sum_{o \in O_j} x_{j,o} = 1, \quad j = 1, \dots, n \quad (3.5)$$

$$x_{j,o} \in \{0, 1\}. \quad (3.6)$$

A Equação 3.3 maximiza a qualidade total preservada, descontada da penalização de transição entre partições adjacentes. A Equação 3.4 impõe o orçamento efetivo de contexto. A Equação 3.5 garante que exatamente uma opção seja escolhida por partição, característica que distingue o núcleo MCKP do problema da mochila clássico.

3.2.7 Solução por Programação Dinâmica

O MCKP com orçamento inteiro admite solução por programação dinâmica. Como a função objetivo inclui uma penalização entre opções adjacentes, o estado também deve registrar a opção escolhida para a última partição. Define-se $D[j, b, o]$ como a qualidade máxima preservada considerando as j primeiras partições sob um orçamento parcial b , quando a opção $o \in O_j$ é escolhida para a partição t_j . A recorrência é apresentada na Equação 3.7.

$$D[j, b, o] = Q_{j,o} + \max_{o' \in O_{j-1}} \left\{ D[j-1, b - c_{j,o}, o'] - \mu d(o', o) \right\}, \quad (3.7)$$

Para a primeira partição, define-se $D[1, b, o] = Q_{1,o}$ quando $c_{1,o} \leq b$, e $D[1, b, o] = -\infty$ caso contrário. A solução ótima corresponde a $\max_{o \in O_n} D[n, B, o]$, e as opções escolhidas são recuperadas por retrocesso (*backtracking*) sobre a tabela de decisão. Para no máximo m opções por partição, a complexidade temporal é $O(nBm^2)$ e a tabela completa ocupa $O(nBm)$ posições; a memória pode ser reduzida quando apenas o valor ótimo, sem o retrocesso, é necessário.

Na implementação de referência, o eixo de orçamento utiliza unidades inteiras de *tokens* (`budget_bucket=1`), preservando a solução exata para o modelo de custos declarado. Uma quantização opcional por blocos permanece disponível como aproximação conservadora para instâncias maiores, mas não é utilizada nos experimentos principais. Os custos das opções incluem a serialização necessária à reconstrução, e o contexto final é recontado antes da chamada ao modelo para verificar a restrição de orçamento.

A Figura 3.2 detalha o fluxo proposto para o preenchimento da tabela de programação dinâmica e para a recuperação das opções selecionadas.

3.2.8 Algoritmo do *Framework*

O funcionamento do *framework* pode ser resumido em cinco etapas encadeadas.

1. **Particionamento.** O *prompt* C é dividido em partições $T = (t_1, \dots, t_n)$ segundo unidades semânticas coerentes.
2. **Estimativa de importância.** Para cada partição, calcula-se a importância $I(t_j; q)$ segundo a Equação 3.1.
3. **Geração de opções.** Para cada partição, aplica-se o conjunto de compressores candidatos, produzindo opções com seus custos $c_{j,o}$ e qualidades $Q_{j,o}$.
4. **Otimização.** Resolve-se a extensão do MCKP das Equações 3.3 a 3.6 por programação dinâmica, respeitando o orçamento B e as restrições de compatibilidade.

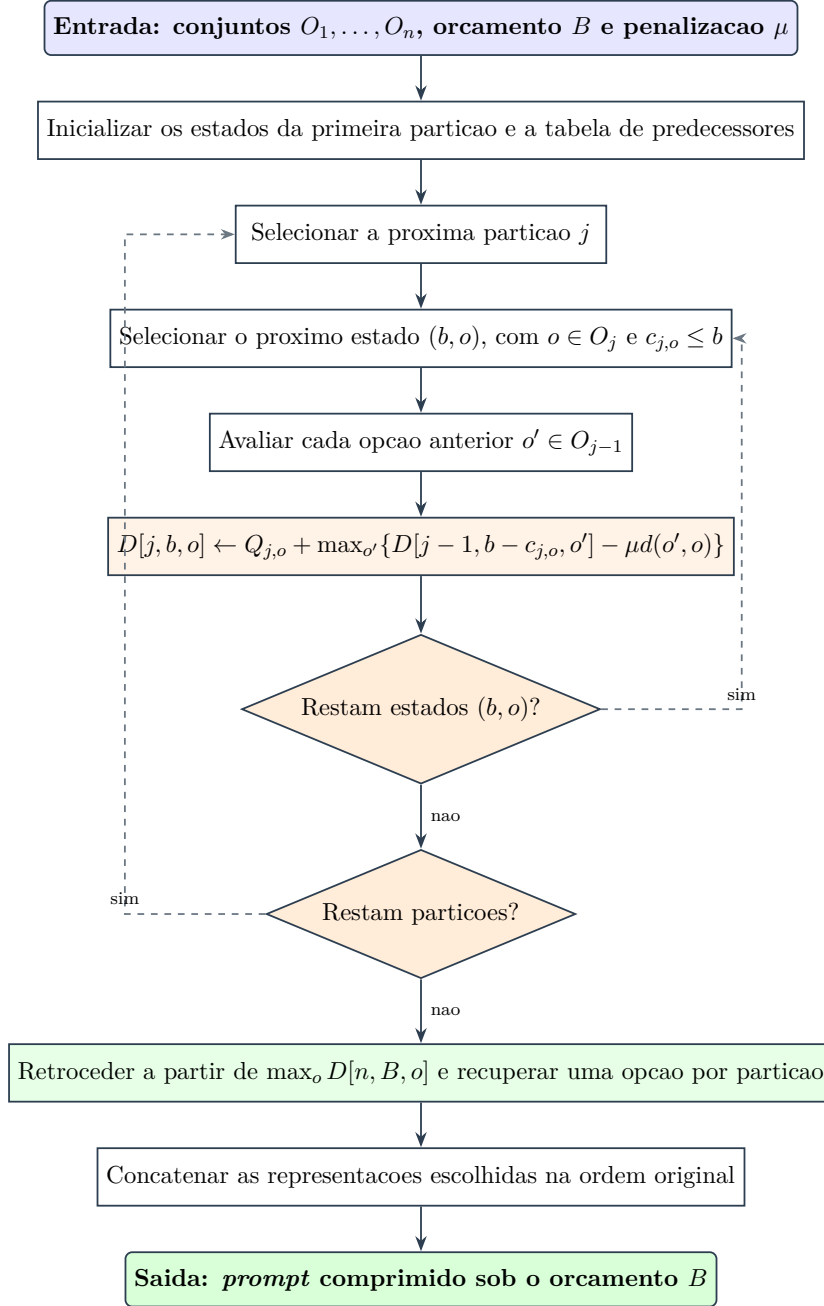


Figura 3.2: Fluxo proposto para resolver a selecao de compressores por programacao dinamica. A tabela registra a melhor qualidade acumulada para cada particao, orcamento parcial e ultima opcao; o retrocesso recupera as opcoes usadas na reconstrucao.

5. **Reconstrução.** As opções escolhidas são concatenadas na ordem original das partições, produzindo o *prompt* comprimido enviado ao modelo.

A Figura 3.3 apresenta a sequência de interação implementada entre os componentes necessários para executar essas etapas.

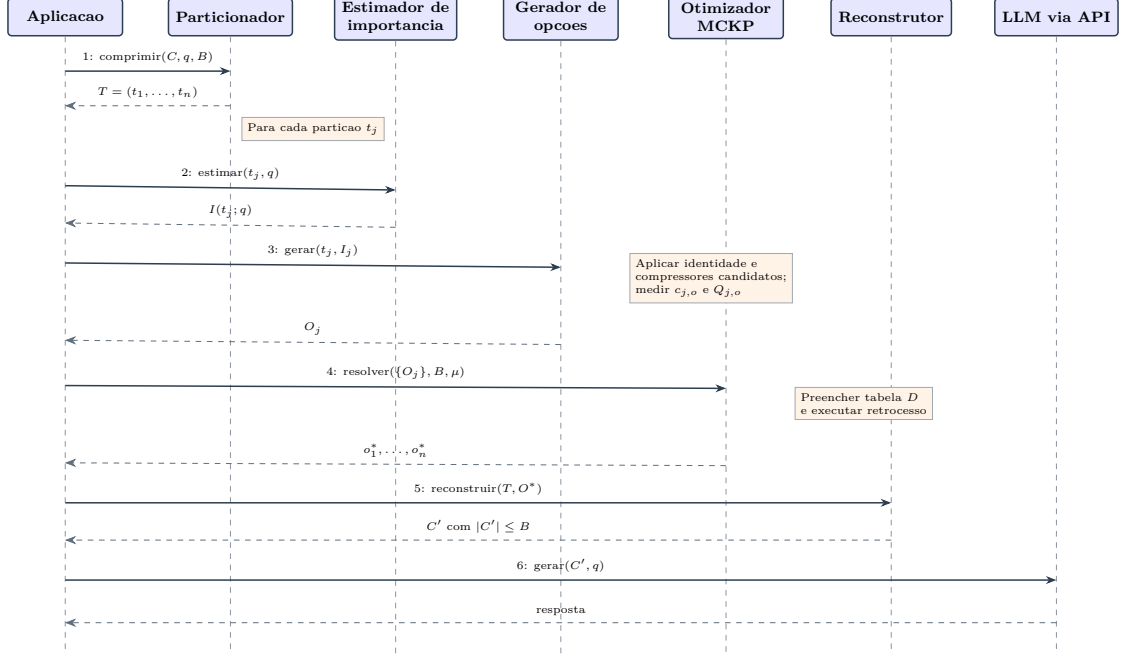


Figura 3.3: Sequencia conceitual da aplicacao do *framework*. Para cada particao, sao calculadas a importancia e as alternativas de compressao; o otimizador seleciona as alternativas globalmente antes da reconstrucao e da chamada ao modelo.

3.2.9 Conjunto de Compressores Compatíveis

O *framework* atua como uma camada de decisão acima de compressores existentes, e não como um novo compressor. Por essa razão, o conjunto de opções por partição é composto por compressores do tipo *text-in/text-out*, isto é, que recebem texto e produzem texto comprimido, o que garante compatibilidade com modelos acessados via API. LLMLingua é empregado como baseline externo, pois constitui o ponto de comparação comum aos três trabalhos posteriores revisados na Seção 2.2.1. LLMLingua-2, Selective Context e CPC são avaliados como candidatos às opções do MCKP. A Tabela 3.7 resume os respectivos papéis experimentais.

A recomendação dos três candidatos se baseia em sua complementaridade. LLMLingua-2 opera em nível de palavra e amortiza o custo de compressão entre consultas, Selective Context introduz uma alternativa sem treinamento específico e CPC-MiniLM preserva sentenças completas com seleção condicionada à pergunta. A escolha por compressores *text-in/text-out* mantém a propriedade não invasiva do

Tabela 3.7: Baseline e compressores candidatos considerados para o MCKP

Compressor	Papel	Característica relevante
Identidade (A_0)	Referência interna	Preserva a partição integralmente, com fidelidade máxima.
LLMLingua-GPT2 [14]	Baseline com <i>scorer</i> adaptado	Implementação do pacote llmlingua , com GPT-2 em CPU para reduzir o custo local.
LLMLingua-2 [36]	Candidato MCKP	Classificação de palavras treinada por destilação, agnóstica à pergunta e com baixa latência.
Selective Context [38]	Candidato MCKP	Remoção de unidades lexicais por auto-informação, sem treinamento específico.
CPC-MiniLM [37]	Aproximação candidata	Seleção de sentenças condicionada à pergunta com <i>embeddings</i> do all-MiniLM-L6-v2.

framework, pois nenhuma etapa exige acesso aos parâmetros internos, retreinamento ou modificação arquitetural do modelo base. O conjunto candidato pode ser estendido com outros compressores compatíveis sem alteração da formulação, uma vez que cada compressor é incorporado apenas como uma nova opção por partição. Nas dez combinações de modelo e orçamento, CPC-MiniLM apresenta a maior média entre os quatro compressores em oito, enquanto LLMLingua-2 lidera nas duas combinações do Qwen 3 30B-A3B. Selective Context preserva a maior fidelidade semântica segundo a métrica adotada. Esses resultados sustentam LLMLingua-2, Selective Context e CPC-MiniLM como conjunto inicial de opções do MCKP; LLMLingua-GPT2 permanece como baseline externo. Os resultados são apresentados na Seção 4.4.

3.2.10 Implementação de Referência

O repositório contém uma implementação integrada no subdiretório `mckp`. O módulo `partitioner.py` produz partições disjuntas; `importance.py` calcula $I(t_j; q)$; `options.py` materializa e avalia as opções; `solver.py` executa a programação dinâmica e o retrocesso; e `reconstructor.py` preserva a ordem das opções selecionadas sem introduzir marcadores não contabilizados. A classe `MCKPStrategy`, em `strategy.py`, coordena essas etapas e expõe a estratégia ao executor de *benchmarks*.

Separação em relação à etapa inicial da matriz local. A estratégia denominada *Semantic Compression* na etapa inicial da matriz local não implementava LLMLingua: ela solicitava que a própria LLM reescrevesse o contexto por meio de uma instrução de compressão, conforme a abordagem de Gilbert et al. [22]. O repositório também continha um `PerplexityCompressor` experimental, que selecionava *tokens* pela perda calculada com GPT-2 e podia ser descrito apenas como inspirado no critério do LLMLingua. Esse protótipo não estava registrado entre as estratégias avaliadas. Para a ampliação da matriz foram criadas entradas explícitas e independentes para LLMLingua, LLMLingua-2, Selective Context e CPC-MiniLM. Assim, resultados da *Semantic Compression* ou do protótipo de perplexidade não são apresentados como resultados de LLMLingua.

Adaptações para execução local. Os experimentos utilizam uma LLM alvo executada localmente, de modo que o compressor e o modelo gerador compartilham os recursos computacionais da mesma máquina. O baseline invoca a classe `PromptCompressor` do pacote `llmlingua 0.2.2`, com `use_llmlingua2=False`, taxa de retenção de 0,5 e execução em CPU. Para reduzir a concorrência de memória com os modelos servidos pelo Ollama, o modelo causal auxiliar é o `gpt2` [61], que já se encontrava no *cache* local e é substancialmente menor que um auxiliar de 7 bilhões de parâmetros. A escolha é compatível com a arquitetura do LLMLingua e com a análise de diferentes modelos auxiliares apresentada por Jiang et al. [14]; entretanto, difere das configurações alinhadas Alpaca-7B e GPT2-Alpaca avaliadas no artigo. Por isso, a configuração é reportada como **LLMLingua-GPT2**: o algoritmo provém da implementação do LLMLingua, enquanto o *scorer* não ajustado e o dispositivo são escolhas locais que devem acompanhar a interpretação dos resultados.

LLMLingua-2 também é executado pelo pacote `llmlingua 0.2.2`, com o `checkpoint microsoft/llmlingua-2-xlm-roberta-large-meetingbank` e `device_map=cpu`. A fixação explícita do dispositivo foi necessária porque a inicialização padrão do pacote pressupunha CUDA, indisponível nessa configuração. A biblioteca Transformers foi fixada na versão 4.46.3, pois o LLMLingua 0.2.2 manipula o formato legado do *cache* de chaves e valores; a versão 5.12.1 instalada inicialmente passou a retornar `DynamicCache` e provocava falhas durante a compressão iterativa. Selective Context é executado pelo pacote `selective-context 0.1.4` com GPT-2, idioma inglês e `reduce_ratio=1-rate`, pois sua API recebe a fração removida, ao passo que o executor comum recebe a fração retida. Essa integração utiliza `spaCy 3.8.14` e o modelo `en_core_web_sm 3.8.0` para o processamento linguístico.

Uma adaptação análoga foi necessária para o CPC. O trabalho original ajusta um Mistral-7B-Instruct com o conjunto CQR para produzir representações de sentenças condicionadas ao contexto [37]. O repositório oficial disponibiliza o código,

o conjunto CQR e compressores pré-treinados baseados em Mistral-7B e Llama-1B [62]; ainda assim, acrescentar outro modelo desse porte à mesma máquina ampliaria a disputa de memória com as LLMs locais avaliadas. Na triagem experimental, o codificador é, portanto, substituído pelo all-MiniLM-L6-v2, da coleção Sentence Transformers no Hugging Face [63]. O compressor divide a partição em sentenças, calcula separadamente os *embeddings* das sentenças e da pergunta, ordena as sentenças pela similaridade de cosseno e preserva as mais relevantes dentro da taxa de retenção, mantendo a ordem textual original. Essa variante é denominada CPC-MiniLM. Ela preserva a seleção extrativa em nível de sentença e o condicionamento à pergunta, mas não incorpora o treinamento contrastivo nem a representação contextual do CPC original. Consequentemente, trata-se de uma aproximação operacional, e diferenças de qualidade e latência entre CPC-MiniLM e os demais compressores não devem ser atribuídas diretamente ao método publicado por Liskavets et al. [37].

Rastreabilidade da triagem. As quatro estratégias adicionadas são executadas somente quando nomeadas explicitamente e não foram incluídas no grupo `all`; essa decisão preserva a reprodutibilidade da etapa inicial da matriz local. A triagem usa o ambiente `.venv`, que contém as dependências dos quatro compressores, e grava seus artefatos em `tests/ollama-local/benchmark_matrix_compressores`, separado de `benchmark_matrix_artigo`. O protocolo completo prevê os cinco modelos, os orçamentos de 8192 e 32768 *tokens*, os sete *benchmarks* e a taxa de retenção de 0,5. O modo `quick` materializa três casos de resposta a perguntas e um caso de sumarização no LongBench, enquanto cada um dos outros seis arquivos locais contém três exemplos, totalizando 22 casos por estratégia e 88 resultados por combinação de modelo e orçamento.

Os artefatos consolidados reúnem as dez combinações de modelo e orçamento, correspondentes aos cinco modelos avaliados com 8192 e 32768 *tokens*. Cada combinação registra 88 execuções, resultantes da aplicação dos quatro compressores aos mesmos 22 casos, totalizando 880 observações. Uma geração de resposta do CPC-MiniLM com o DeepSeek-R1 32B em 8192 *tokens*, no caso `meeting_summarization_0`, terminou por *timeout*. A saída de erro não foi tratada como resposta e foi excluída das médias de pontuação e latência, de modo que essas duas métricas reúnem 879 respostas válidas; a taxa retida e a fidelidade dessa execução permanecem válidas, pois a compressão foi concluída antes da chamada ao modelo alvo. A execução utiliza um processo por combinação de modelo e orçamento, no qual as quatro estratégias são registradas conjuntamente; assim, GPT-2, XLM-RoBERTa, o segundo GPT-2 e MiniLM podem coexistir durante a rodada. O processo é encerrado antes da combinação seguinte, liberando o modelo alvo e os auxiliares. Essa organização preserva a comparabilidade de qualidade dentro de

cada combinação, mas as latências devem ser interpretadas como custo descritivo de ponta a ponta no ambiente local, e não como medição isolada do custo de cada compressor.

O orçamento é derivado independentemente para as janelas de 8192 e 32768 *tokens*. Em cada rodada são descontados o *template* completo, a pergunta, a reserva de saída e uma margem de segurança para diferenças entre o contador *tiktoken* usado nas métricas e o tokenizador do modelo alvo. A implementação registra o custo previsto pelo resolvedor, o custo recontado do contexto final, o orçamento não utilizado e eventuais compressores indisponíveis, permitindo auditar cada execução.

3.2.11 Aplicação da Solução de Ponta a Ponta

«TODO-SOLUCAO»

Esta subseção apresentará a aplicação da solução sobre um caso concreto, ilustrando o *prompt* de entrada, o particionamento resultante, as opções de compressão avaliadas por partição, a alocação de orçamento escolhida pelo MCKP e o *prompt* comprimido final enviado ao modelo, de forma a tornar explícito o funcionamento completo do *framework*.

3.3 Perguntas de Pesquisa e Hipóteses

A avaliação do *framework* é orientada pelas seguintes perguntas de pesquisa.

- **PP1.** A alocação seletiva de compressão por partição preserva mais informação útil do que a compressão uniforme sob o mesmo orçamento de *tokens*?
- **PP2.** A função de importância, ao combinar relevância à tarefa, densidade de informação e saliência posicional, melhora a qualidade das respostas em comparação com uma alocação de orçamento independente da posição?
- **PP3.** A penalização de transição entre partições adjacentes melhora a coerência do *prompt* reconstruído sem prejudicar a acurácia final?
- **PP4.** Como o *framework* se comporta em diferentes escalas de modelo e orçamentos de contexto?

A partir dessas perguntas, formulam-se três hipóteses de trabalho.

- **H1.** Sob um orçamento fixo, a alocação seletiva por partição preserva mais informação relevante para a tarefa do que a compressão uniforme, resultando em acurácia igual ou superior nos *benchmarks* de contexto longo.

- **H2.** A inclusão da saliência posicional na função de importância reduz o efeito do viés *lost in the middle*, ao proteger regiões centrais de alto valor da compressão excessiva.
- **H3.** A penalização de transição reduz artefatos de incoerência entre trechos adjacentes sem perda significativa de acurácia.

As hipóteses definem critérios de avaliação e não são tratadas como resultados já obtidos.

3.4 Plano Experimental para Avaliação do *Framework*

A avaliação do *framework* é organizada em quatro componentes.

1. **Benchmarks.** Utilização da mesma suíte de contexto longo empregada na validação empírica, descrita na Seção 3.1.1, para manter a comparabilidade com a matriz de técnicas uniformes.
2. **Baselines.** Comparação com a estratégia *Raw*, com a *Sliding Window* uniforme e com a aplicação uniforme de cada compressor candidato, isolando o efeito da alocação seletiva.
3. **Métricas.** Registro de acurácia, latência e taxa de compressão efetiva, complementadas por medidas de preservação de informação e fidelidade semântica.
4. **Estudos de ablação.** Remoção sucessiva de componentes, incluindo a saliência posicional da função de importância e a penalização de transição, para estimar a contribuição individual de cada componente.

O plano prioriza a preservação de informação porque o objetivo do *framework* é alocar o orçamento segundo a relevância das partições, e não apenas reduzir o número total de *tokens*. Sua execução completa permanece como trabalho futuro no Capítulo 5.

Capítulo 4

Resultados e Discussão

Este capítulo interpreta os resultados da avaliação empírica descrita no Capítulo 3. A matriz compara nove técnicas de tratamento de contexto em cinco modelos e dois orçamentos. As cinco técnicas da etapa inicial e os quatro compressores incorporados posteriormente foram executados nas dez combinações de modelo e orçamento. A triagem dos compressores registrou 880 execuções; uma chamada ao DeepSeek-R1 32B terminou por *timeout*, de modo que as agregações dependentes da resposta consideram 879 observações válidas.

4.1 Escopo da Evidência Experimental

As Tabelas 3.4 e 3.5 organizam as nove técnicas como linhas de uma única matriz, agrupadas pelo modelo alvo. Os resultados de *Raw*, *Sliding Window*, *Parallel Window*, *Semantic Compression*, RIG, LLMLingua-GPT2, LLMLingua-2, Selective Context e CPC-MiniLM estão disponíveis para os cinco modelos nos orçamentos de 8192 e 32768 *tokens*. No bloco CPC-MiniLM, DeepSeek-R1 32B e 8192 *tokens*, a pontuação do QMSum e a média macro excluem a observação encerrada por *timeout* e são identificadas por um asterisco.

Cada célula das duas matrizes corresponde à pontuação agregada no respectivo *benchmark*. A última coluna atribui o mesmo peso às sete tarefas e, portanto, constitui uma média macro por *benchmark*. Na análise operacional dos quatro compressores, também é apresentada a média calculada diretamente sobre os 22 casos de cada combinação. Como essa segunda estatística pondera os *benchmarks* pelo número de casos, seus valores não precisam coincidir com a média macro da matriz principal.

4.2 Desempenho na Matriz Multimodelo

4.2.1 Efeito do Orçamento de Contexto

Na etapa inicial, a *Sliding Window* apresenta a maior média sob o orçamento de 8192 *tokens* no Llama 3.1 8B, no Gemma 4 26B e no GPT-OSS 20B, com 0.626, 0.573 e 0.600, respectivamente. A RIG lidera no Qwen 3 30B-A3B, com 0.521, enquanto a estratégia *Raw* lidera no DeepSeek-R1 32B, com 0.673. Nesse orçamento, portanto, uma técnica que trata o contexto supera o envio direto em quatro dos cinco modelos da matriz inicial.

Sob o orçamento de 32768 *tokens*, a relação se altera. Entre as cinco técnicas da etapa inicial, a estratégia *Raw* lidera no Llama 3.1 8B, no Gemma 4 26B e no DeepSeek-R1 32B, com 0.627, 0.570 e 0.708. A RIG permanece à frente no Qwen 3 30B-A3B, com 0.521, enquanto a *Sliding Window* mantém o maior resultado no GPT-OSS 20B, com 0.647. A ampliação do orçamento beneficia particularmente a estratégia *Raw* no GPT-OSS 20B, cuja média passa de 0.530 para 0.637, e no DeepSeek-R1 32B, em que aumenta de 0.673 para 0.708. No conjunto ampliado de nove técnicas, CPC-MiniLM alcança 0.573 no Gemma 4 26B e supera por 0.003 a estratégia *Raw*. Essa combinação constitui a única liderança de um dos quatro compressores adicionados na matriz completa.

Os resultados indicam que a utilidade da segmentação varia com a pressão exercida pelo orçamento e com o modelo alvo. Quando o limite é menor, a seleção de trechos pode evitar parte da perda causada pelo truncamento do contexto integral. Quando o orçamento acomoda uma fração maior da entrada, essa vantagem diminui no Llama 3.1 8B e entre as cinco técnicas originalmente avaliadas no Gemma 4 26B. O comportamento do Qwen 3 30B-A3B, do GPT-OSS 20B e do próprio CPC-MiniLM no Gemma 4 26B impede, contudo, que essa relação seja generalizada independentemente da técnica e do modelo.

4.2.2 Variação entre Modelos e Tarefas

A média macro não identifica uma técnica dominante em todas as tarefas. O TriviaQA atinge 1.000 em grande parte das combinações da matriz original e, por isso, apresenta menor poder discriminante nesta amostra. Em contraste, o Natural Questions varia de forma acentuada. Sob 8192 *tokens*, por exemplo, a *Semantic Compression* obtém 0.691 no Llama 3.1 8B e 0.000 no Gemma 4 26B. A mesma técnica não lidera a média global em nenhum modelo, mostrando que o desempenho elevado em uma tarefa isolada não implica estabilidade na suíte.

Também se observam diferenças entre modelos de uma mesma categoria ampla. No Qwen 3 30B-A3B, a RIG apresenta as maiores médias globais nos dois orçamen-

tos; no GPT-OSS 20B, essa posição pertence à *Sliding Window*. Entre os modelos densos, o Llama 3.1 8B favorece a *Sliding Window* em 8192 *tokens* e a estratégia *Raw* em 32768 *tokens*, enquanto o Gemma 4 26B passa da *Sliding Window* para CPC-MiniLM. Essas observações caracterizam a matriz avaliada, mas não isolam o efeito causal da arquitetura, pois tamanho, treinamento, quantização e implementação também variam entre os modelos.

4.3 Relação entre Qualidade, Compressão e Latência

A Tabela 3.6 evidencia que redução de contexto, qualidade e custo não produzem uma ordenação única. Entre as cinco técnicas da etapa inicial, a RIG apresenta a menor latência entre as que tratam o contexto em todos os modelos sob 8192 *tokens* e em quatro dos cinco modelos sob 32768 *tokens*. Sua taxa retida é de aproximadamente 0.692, mas essa vantagem de tempo não é acompanhada pela maior média de qualidade na maior parte da matriz.

A *Semantic Compression* retém entre 0.328 e 0.417 do contexto e constitui a redução mais agressiva entre as técnicas da etapa inicial. Como exige chamadas adicionais ao modelo alvo, também apresenta as maiores latências nos cinco modelos. No Gemma 4 26B, a média alcança 306142 ms em 8192 *tokens* e 354957 ms em 32768 *tokens*. A técnica ainda exibe forte variação entre tarefas, de modo que a maior redução de *tokens* não se traduz em melhor desempenho final.

A *Sliding Window* retém aproximadamente 0.857 do contexto e ocupa uma posição intermediária de custo. Sua escolha como estratégia base decorre da regularidade observada na etapa inicial: em sete das dez combinações de modelo e orçamento, ela apresenta a maior média entre as técnicas que tratam o contexto. Esse resultado não estabelece uma solução universal, pois RIG lidera nas duas combinações do Qwen 3 30B-A3B e CPC-MiniLM supera toda a matriz no Gemma 4 26B sob 32768 *tokens*.

4.4 Análise dos Novos Compressores

LLMLingua-GPT2, LLMLingua-2, Selective Context e CPC-MiniLM foram avaliados com taxa nominal de retenção de 0,5. Para cada combinação de modelo e orçamento, as quatro técnicas processaram os mesmos 22 casos: quatro do Long-Bench e três de cada um dos outros seis *benchmarks*. Os dez artefatos consolidados registram 88 execuções cada, totalizando 880 observações. Uma delas não produziu resposta válida, conforme descrito na Seção 4.1; as comparações de pontuação e latência utilizam as 879 respostas restantes.

No Llama 3.1 8B, CPC-MiniLM apresenta a maior média entre os quatro compressores nos dois orçamentos, com 0.531 em ambos. Em 8192 *tokens*, LLMLingua-2, Selective Context e LLMLingua-GPT2 obtêm 0.492, 0.480 e 0.379; em 32768 *tokens*, as médias correspondentes são 0.503, 0.487 e 0.367. Embora CPC-MiniLM lidere o grupo, permanece abaixo da *Sliding Window*, com 0.626 em 8192 *tokens*, e da estratégia *Raw*, com 0.627 em 32768 *tokens*.

No Gemma 4 26B, CPC-MiniLM também ocupa a primeira posição entre os compressores, com 0.547 e 0.573 nos dois orçamentos. LLMLingua-2 alcança 0.457 e 0.467, Selective Context obtém 0.353 e 0.336, e LLMLingua-GPT2 registra 0.274 e 0.292. O resultado de 0.573 do CPC-MiniLM em 32768 *tokens* supera as oito alternativas da matriz, ainda que por margem reduzida em relação aos 0.570 da estratégia *Raw*.

No Qwen 3 30B-A3B, a ordenação se inverte. LLMLingua-2 lidera entre os compressores com 0.430 em 8192 *tokens* e 0.413 em 32768 *tokens*. CPC-MiniLM permanece em segundo lugar, com 0.394 nos dois orçamentos, seguido por Selective Context, com 0.258 e 0.282, e LLMLingua-GPT2, com 0.206 e 0.190. Nenhum dos quatro supera a RIG, que obtém 0.521 nos dois orçamentos.

No DeepSeek-R1 32B, CPC-MiniLM lidera o grupo com médias de 0.613 e 0.634 nos orçamentos de 8192 e 32768 *tokens*. LLMLingua-2 obtém 0.581 e 0.578, enquanto Selective Context alcança 0.576 e 0.585. LLMLingua-GPT2 permanece abaixo dos demais, com 0.321 e 0.333. CPC-MiniLM também ocupa a segunda posição entre as nove técnicas nos dois orçamentos, atrás da estratégia *Raw*. O resultado de 0.613 deve ser interpretado como provisório, pois sua média de QMSum utiliza duas respostas válidas.

No GPT-OSS 20B, CPC-MiniLM também apresenta a maior média entre os compressores, com 0.585 em 8192 *tokens* e 0.537 em 32768 *tokens*. LLMLingua-2 registra 0.487 e 0.438, Selective Context obtém 0.427 e 0.405, e LLMLingua-GPT2 alcança 0.290 nos dois orçamentos. Em 8192 *tokens*, CPC-MiniLM fica 0.001 abaixo da RIG e 0.015 abaixo da *Sliding Window*; em 32768 *tokens*, a diferença para a *Sliding Window* aumenta para 0.110. Consideradas as dez combinações, CPC-MiniLM lidera o grupo em oito, e LLMLingua-2 lidera nas duas combinações do Qwen.

O detalhamento por tarefa explica parte dessa mudança de ordenação. LLMLingua-2 alcança as maiores pontuações de ZeroSCROLLS entre todas as técnicas no Gemma 4 26B e no Qwen 3 30B-A3B sob 8192 *tokens*, com 0.583 e 0.592. Selective Context, por sua vez, alcança 0.656 no ZeroSCROLLS do Llama 3.1 8B sob 32768 *tokens*. CPC-MiniLM obtém 0.743 e 0.827 no ZeroSCROLLS do DeepSeek-R1 32B e 0.707 no GPT-OSS 20B sob 8192 *tokens*, os maiores valores das respectivas colunas. Assim, a média agregada decorre de perfis distintos por tarefa e por modelo,

e não de uma vantagem uniforme de um único compressor.

A Tabela 4.1 complementa a matriz principal com a pontuação média por caso, a taxa efetivamente retida, a fidelidade semântica e a latência total. A fidelidade mede a cobertura do contexto original por meio de *embeddings* do all-MiniLM-L6-v2, enquanto a latência inclui a compressão e a geração da resposta pelo modelo alvo. Na coluna de taxa retida, valores menores representam compressão mais agressiva. O asterisco identifica as métricas de pontuação e latência calculadas após a exclusão da chamada encerrada por *timeout*; as métricas de compressão dessa chamada permanecem incluídas.

Tabela 4.1: Métricas operacionais dos quatro compressores nas dez combinações de modelo e orçamento

Técnica	Contexto	Pontuação	Taxa retida	Fidelidade	Latência (ms)
<i>Llama 3.1 8B</i>					
LLMLingua-GPT2	8192	0.393	0.503	0.613	8451
LLMLingua-2	8192	0.506	0.457	0.666	9484
Selective Context	8192	0.489	0.548	0.732	10344
CPC-MiniLM	8192	0.538	0.517	0.678	6736
LLMLingua-GPT2	32768	0.381	0.512	0.616	10127
LLMLingua-2	32768	0.517	0.482	0.673	12385
Selective Context	32768	0.496	0.584	0.738	14825
CPC-MiniLM	32768	0.538	0.531	0.679	10345
<i>Gemma 4 26B</i>					
LLMLingua-GPT2	8192	0.273	0.503	0.613	18237
LLMLingua-2	8192	0.479	0.457	0.666	17477
Selective Context	8192	0.369	0.548	0.732	19508
CPC-MiniLM	8192	0.554	0.517	0.678	14307
LLMLingua-GPT2	32768	0.296	0.512	0.616	18864
LLMLingua-2	32768	0.489	0.482	0.673	20048
Selective Context	32768	0.352	0.584	0.738	22037
CPC-MiniLM	32768	0.578	0.531	0.679	15755
<i>Qwen 3 30B-A3B</i>					
LLMLingua-GPT2	8192	0.197	0.503	0.613	14144
LLMLingua-2	8192	0.444	0.457	0.666	14503
Selective Context	8192	0.269	0.548	0.732	16140
CPC-MiniLM	8192	0.398	0.517	0.678	11840
LLMLingua-GPT2	32768	0.182	0.512	0.616	17128
LLMLingua-2	32768	0.428	0.482	0.673	18423
Selective Context	32768	0.292	0.584	0.738	21096
CPC-MiniLM	32768	0.398	0.531	0.679	15836

Continua na próxima página

Tabela 4.1 – continuação

Técnica	Contexto	Pontuação	Taxa retida	Fidelidade	Latência (ms)
<i>DeepSeek-R1 32B</i>					
LLMLingua-GPT2	8192	0.329	0.503	0.613	72195
LLMLingua-2	8192	0.577	0.457	0.666	78809
Selective Context	8192	0.573	0.548	0.732	108964
CPC-MiniLM	8192	0.623*	0.517	0.678	75366*
LLMLingua-GPT2	32768	0.341	0.512	0.616	80249
LLMLingua-2	32768	0.575	0.482	0.673	90642
Selective Context	32768	0.581	0.584	0.738	89828
CPC-MiniLM	32768	0.628	0.531	0.679	83529
<i>GPT-OSS 20B</i>					
LLMLingua-GPT2	8192	0.296	0.503	0.613	15449
LLMLingua-2	8192	0.507	0.457	0.666	14546
Selective Context	8192	0.421	0.548	0.732	15940
CPC-MiniLM	8192	0.589	0.517	0.678	9214
LLMLingua-GPT2	32768	0.296	0.512	0.616	16308
LLMLingua-2	32768	0.461	0.482	0.673	15648
Selective Context	32768	0.401	0.584	0.738	17707
CPC-MiniLM	32768	0.544	0.531	0.679	11079

O padrão operacional é consistente nas dez combinações. LLMLingua-2 produz a menor taxa retida em todas elas, com 0.457 em 8192 *tokens* e 0.482 em 32768 *tokens*. Selective Context apresenta a maior fidelidade semântica, com 0.732 e 0.738. CPC-MiniLM alcança a maior pontuação média por caso em oito combinações e a menor latência entre os compressores em sete. LLMLingua-GPT2 é o compressor mais rápido no Llama 3.1 8B sob 32768 *tokens* e nas duas combinações do DeepSeek-R1 32B. No caso marcado por asterisco, a pontuação e a latência do CPC-MiniLM baseiam-se em 21 respostas válidas, enquanto as taxas de retenção e fidelidade consideram as 22 compressões concluídas. Esses resultados mostram que qualidade, redução de *tokens*, cobertura semântica e latência favorecem métodos diferentes.

As latências devem ser interpretadas com cautela. Cada combinação de modelo e orçamento foi executada em um único processo que registrou os quatro compressores, permitindo a coexistência dos respectivos modelos auxiliares durante a rodada. Os valores representam, portanto, o custo de ponta a ponta observado no ambiente local e são comparáveis de forma descritiva dentro desse protocolo; eles não isolam o custo de compressão nem reproduzem diretamente as medições de hardware dos artigos originais.

4.5 Seleção das Opções do MCKP

Os resultados sustentam uma seleção inicial, e não definitiva, de LLMLingua-2, Selective Context e CPC-MiniLM como opções de compressão do MCKP. CPC-MiniLM apresenta a maior média do grupo no Llama 3.1 8B, no Gemma 4 26B, no DeepSeek-R1 32B e no GPT-OSS 20B, enquanto LLMLingua-2 lidera no Qwen 3 30B-A3B. Além da qualidade por tarefa, os três métodos preservam propriedades complementares: LLMLingua-2 realiza a compressão mais agressiva, Selective Context mantém a maior fidelidade semântica segundo a métrica utilizada e CPC-MiniLM associa seleção condicionada à pergunta a resultados mais altos em oito das dez combinações. LLMLingua-GPT2 permanece como baseline externo, pois apresenta a menor média do grupo em todas as combinações.

Essa seleção deve ser interpretada à luz das adaptações descritas na Seção 3.2.10. LLMLingua-GPT2 utiliza GPT-2 como modelo auxiliar para viabilizar sua execução ao lado de uma LLM local. CPC-MiniLM substitui o codificador do método publicado pelo all-MiniLM-L6-v2 disponível no Hugging Face. Os resultados caracterizam, portanto, essas configurações experimentais e não constituem uma reprodução direta do desempenho das configurações canônicas descritas nos artigos originais.

A heterogeneidade entre modelos e tarefas fornece a motivação empírica para manter múltiplas opções no MCKP, mas não demonstra, por si só, que a alocação seletiva por partição supera a aplicação uniforme dos compressores. Essa hipótese depende da execução específica do *framework*, de comparações sob o mesmo orçamento e de estudos de ablação da função de importância e da penalização de transição. A triagem seleciona e caracteriza as opções candidatas; ela não equivale à validação do resolvidor MCKP.

4.6 Limitações da Análise

A primeira limitação é o tamanho da triagem, composta por 22 casos em cada combinação de modelo e orçamento e sem repetições independentes. A média macro reduz o efeito do número desigual de casos entre tarefas, mas não substitui uma avaliação em escala maior nem quantifica a incerteza das diferenças observadas. Além disso, uma chamada do CPC-MiniLM no DeepSeek-R1 32B sob 8192 *tokens* terminou por *timeout*; por isso, sua pontuação agregada utiliza 21 respostas e a média de QMSum utiliza somente duas. Margens reduzidas, como a diferença de 0.003 entre CPC-MiniLM e *Raw* no Gemma 4 26B sob 32768 *tokens*, devem ser interpretadas de forma descritiva.

A segunda limitação está associada às adaptações locais. O *scorer* GPT-2 do LLMLingua-GPT2 difere das configurações alinhadas avaliadas no trabalho origi-

nal, e CPC-MiniLM não incorpora o treinamento contrastivo nem a representação contextual do CPC. A métrica de fidelidade utiliza o mesmo codificador MiniLM empregado por CPC-MiniLM e, portanto, não é inteiramente independente dessa técnica. Por essa razão, a pontuação nas tarefas é adotada como critério principal, enquanto a fidelidade atua apenas como evidência complementar.

A terceira limitação é que a triagem compara compressores aplicados uniformemente a cada entrada, mas não avalia a política seletiva proposta. O *framework* MCKP ainda requer execução sob os mesmos orçamentos, comparação com as melhores estratégias uniformes e estudos de ablação da função de importância e da penalização de transição. Assim, a heterogeneidade observada fundamenta a escolha das opções, mas não demonstra superioridade da alocação por partição.

4.7 Síntese dos Resultados

Três conclusões delimitadas pela evidência podem ser extraídas. Primeiro, nenhuma técnica domina simultaneamente todos os modelos, tarefas e orçamentos. A *Sliding Window* apresenta o melhor equilíbrio recorrente entre as cinco técnicas da etapa inicial, enquanto a estratégia *Raw* se torna mais competitiva com a ampliação do orçamento. Segundo, redução de *tokens*, fidelidade, latência e qualidade final não são objetivos equivalentes: LLMLingua-2 comprime mais, Selective Context preserva maior similaridade semântica e CPC-MiniLM alcança as maiores pontuações entre os compressores em oito das dez combinações. Terceiro, o Qwen 3 30B-A3B altera essa ordenação em favor de LLMLingua-2, e o CPC-MiniLM supera toda a matriz apenas no Gemma 4 26B sob 32768 *tokens*. Em conjunto, os resultados justificam manter opções de compressão com perfis distintos no espaço de decisão do MCKP. A ausência da avaliação completa do resolvidor, porém, impede concluir que a alocação seletiva supera a melhor aplicação uniforme dessas opções.

Capítulo 5

Conclusão e Trabalhos Futuros

5.1 Síntese do Trabalho

Esta dissertação investigou o problema da janela de contexto em Modelos de Linguagem de Grande Escala a partir de uma perspectiva não invasiva, isto é, aplicável a modelos acessados via API, sem retreinamento nem modificação arquitetural. O trabalho partiu da constatação de que janelas de contexto nominalmente grandes frequentemente não são utilizadas de forma eficaz pelos modelos, em razão da complexidade quadrática da atenção e de fenômenos como o viés *lost in the middle*, o *context rot* e o colapso de atenção.

O Capítulo 2 revisou os fundamentos técnicos dos LLMs e organizou as estratégias não invasivas de extensão de contexto em uma taxonomia, composta por compressão de *prompt*, segmentação de contexto e janela deslizante, métodos baseados em *MapReduce* e alocação de memória externa. A revisão identificou como lacuna recorrente o tratamento uniforme do *prompt*, em que um único método e uma única taxa de compressão são aplicados a toda a entrada, desconsiderando sua estrutura interna heterogênea.

O Capítulo 3 apresentou a validação empírica de cinco estratégias não invasivas em modelos de arquiteturas distintas, que reúnem redes densas, redes de mistura de especialistas e modelos ajustados para raciocínio, sobre uma suíte de *benchmarks* de contexto longo. A estratégia *Sliding Window* apresentou o melhor equilíbrio recorrente entre acurácia e latência na etapa inicial, o que sustentou sua adoção como estratégia base. A triagem adicional dos compressores abrangeu os cinco modelos e os dois orçamentos, totalizando 880 execuções e 879 respostas válidas. CPC-MiniLM obteve as maiores médias entre os quatro compressores em oito das dez combinações, enquanto LLMLingua-2 liderou nas duas combinações do Qwen 3 30B-A3B. Selective Context, por sua vez, apresentou a maior fidelidade semântica segundo a métrica adotada. Esses perfis complementares sustentaram a incorporação dos três

ao conjunto inicial de opções do MCKP. Em seguida, o mesmo capítulo formalizou o *framework* proposto, que descreve a compressão seletiva de contexto como um problema da mochila de múltipla escolha, com função de importância, modelo de qualidade, restrições de compatibilidade e solução por programação dinâmica.

O Capítulo 4 discutiu os resultados empíricos, analisou a relação entre qualidade, compressão e latência e delimitou as implicações da triagem para o conjunto de opções do *framework*, sem tratar a caracterização dos compressores como evidência de superioridade do MCKP.

5.2 Contribuições

O trabalho ofereceu quatro contribuições principais. A primeira é uma revisão sistemática de técnicas não invasivas de extensão de contexto, organizada em uma taxonomia por lógica operacional. A segunda é uma avaliação empírica comparativa de cinco estratégias em modelos de arquiteturas distintas, complementada pela triagem de quatro configurações de compressão nos cinco modelos, reportando desempenho, latência, taxa retida e fidelidade semântica. A terceira é a formalização de um *framework* de compressão seletiva via problema da mochila, que atua como uma camada de decisão acima de compressores existentes. A quarta é um plano experimental reproduzível para a avaliação do *framework*, orientado por métricas de preservação de informação e não apenas de desempenho final.

5.3 Limitações

O trabalho apresenta limitações que devem ser consideradas na interpretação dos resultados. A triagem dos quatro compressores utiliza 22 casos por combinação, sem repetições independentes, e uma chamada do CPC-MiniLM no DeepSeek-R1 32B sob 8192 *tokens* terminou por *timeout*; essa combinação possui 21 respostas válidas. A fidelidade semântica adicionada à triagem mede cobertura por *embeddings*, mas não captura integralmente o risco de alucinação por perda de conteúdo. No caso de CPC-MiniLM, essa métrica usa o mesmo codificador empregado pela aproximação, não constituindo uma avaliação inteiramente independente. Além disso, o *framework* proposto foi formalizado e teve seu plano de avaliação definido, mas sua verificação empírica completa não foi executada nesta dissertação, o que significa que as hipóteses de trabalho permanecem como direções a serem testadas.

5.4 Trabalhos Futuros

A partir das limitações identificadas, delineiam-se as seguintes direções de trabalho futuro. A primeira é repetir a chamada encerrada por *timeout* e executar o plano experimental descrito na Seção 3.4, incluindo a comparação do MCKP com as melhores estratégias uniformes e os estudos de ablação sobre a função de importância e a penalização de transição. A segunda é incorporar métricas adicionais de preservação de informação e fidelidade semântica, preferencialmente independentes dos codificadores usados pelos compressores, para avaliar o *framework* para além do desempenho final. A terceira é avaliar CPC com os *checkpoints* disponibilizados pelo trabalho original, separando o custo dessa configuração dos resultados da aproximação CPC-MiniLM. A quarta é estender o *framework* a modelos multimodais, em que partições podem conter texto, imagens ou outros tipos de conteúdo, com compressores específicos por modalidade. A quinta é otimizar a latência do próprio *framework*, uma vez que as etapas de estimativa de importância e de geração de opções introduzem custo computacional que precisa ser compatível com aplicações interativas.

De forma mais ampla, o trabalho sugere que a compressão de contexto em LLMs pode ser tratada como um problema de alocação de recursos, e não como uma redução uniforme de *tokens*. Essa formulação abre espaço para políticas de compressão adaptativas, que respondem à estrutura interna do *prompt* e ao orçamento disponível, e que podem ser aplicadas de forma não invasiva sobre modelos já treinados.

Referências Bibliográficas

- [1] MIKOLOV, T., CHEN, K., CORRADO, G., et al. “Efficient Estimation of Word Representations in Vector Space”, *arXiv preprint arXiv:1301.3781*, 2013.
- [2] VASWANI, A., SHAZEER, N., PARMAR, N., et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*, v. 30, 2017.
- [3] DEVLIN, J., CHANG, M.-W., LEE, K., et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.
- [4] RADFORD, A., NARASIMHAN, K., SALIMANS, T., et al. *Improving Language Understanding by Generative Pre-Training*. Relatório técnico, OpenAI, 2018.
- [5] RUDER, S., PETERS, M. E., SWAYAMDIPTA, S., et al. “Transfer Learning in Natural Language Processing”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials*, pp. 15–18, 2019.
- [6] BROWN, T. B., MANN, B., RYDER, N., et al. “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems*, v. 33, pp. 1877–1901, 2020.
- [7] HUYNH, N., LIN, B. “Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications”, *arXiv preprint arXiv:2503.01245*, 2025. doi: 10.48550/arXiv.2503.01245.
- [8] SCARLATOS, A., LIU, N., LEE, J., et al. “Training LLM-Based Tutors to Improve Student Learning Outcomes in Dialogues”. In: *Artificial Intelligence in Education*, pp. 251–266. Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-98414-3_18.

- [9] HUANG, Y., XU, J., LAI, J., et al. “Advancing Transformer Architecture in Long-Context Large Language Models: A Comprehensive Survey”. 2024. arXiv:2311.12351.
- [10] REID, M., SAVINOV, N., TEPLYASHIN, D., et al. *Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context*. Relatório técnico, Google DeepMind, 2024.
- [11] CHROMA. “The Curse of the Long Context: Why Your RAG System is Failing”. <https://www.trychroma.com/blog>, 2024.
- [12] CHEN, S., LIN, Z., POLYANSKIY, Y., et al. “Critical Attention Scaling in Long-Context Transformers”. 2025. arXiv:2510.05554.
- [13] LIU, N. F., LIN, K., HEWITT, J., et al. “Lost in the Middle: How Language Models Use Long Contexts”, *Transactions of the Association for Computational Linguistics*, v. 12, pp. 157–173, 2024.
- [14] JIANG, H., WU, Q., LIN, C.-Y., et al. “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13358–13376, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.825. Disponível em: <<https://aclanthology.org/2023.emnlp-main.825/>>.
- [15] AMAN, S., RESEARCHERS, A. “Latency Analysis in Long-Context LLM Applications”, *AI Performance Review*, 2024.
- [16] YU, T., XU, A., AKKIRAJU, R. “In Defense of RAG in the Era of Long-Context Language Models”. 2024. arXiv:2409.01666.
- [17] WANG, X., SALMANI, M., OMIDI, P., et al. “Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models”. 2024. arXiv:2402.02244.
- [18] ZHAO, W. X., ZHOU, K., LI, J., et al. “A Survey of Large Language Models”, *arXiv preprint arXiv:2303.18223*, 2023.
- [19] NAVEED, H., KHAN, A. U., QIU, S., et al. “A Comprehensive Overview of Large Language Models”, *arXiv preprint arXiv:2307.06435*, 2023.
- [20] WAN, Z., WANG, X., LIU, C., et al. “Efficient Large Language Models: A Survey”, *arXiv preprint arXiv:2312.03863*, 2023.

- [21] ZHUANG, B., LIU, J., PAN, Z., et al. “A Survey on Efficient Training of Transformers”. 2023. arXiv:2302.01107.
- [22] GILBERT, H., SANDBORN, M., SCHMIDT, D. C., et al. “Semantic Compression With Large Language Models”. 2023. arXiv:2304.12512.
- [23] KAMRADT, G. “Workaround OpenAI’s Token Limit With Chain Types”. https://www.youtube.com/watch?v=f9_BWhCI4Zo, 2023.
- [24] PACKER, C., WOODERS, S., LIN, K., et al. “MemGPT: Towards LLMs as Operating Systems”. 2024. arXiv:2310.08560.
- [25] XIAO, C., ZHANG, P., HAN, X., et al. “InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory”. 2024. arXiv:2402.04617.
- [26] LEWIS, P., PEREZ, E., PIKTUS, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*, v. 33, pp. 9459–9474, 2020.
- [27] SHARMA, C. “Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers”. 2025. arXiv:2506.00054.
- [28] PICKETT, M., HARTMAN, J., BHOWMICK, A. K., et al. “Better RAG using Relevant Information Gain”. 2024. arXiv:2407.12101.
- [29] ALONSO, N., MILLIDGE, B. “Mixture-of-PageRanks: Replacing Long-Context with Real-Time, Sparse GraphRAG”. 2024. arXiv:2412.06078.
- [30] KWON, W., LI, Z., ZHUANG, S., et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. 2023. arXiv:2309.06180.
- [31] CHOI, Y., KIM, S., BASSOLE, Y. C. F., et al. “Enhanced Retrieval-Augmented Generation Using Low-Rank Adaptation”, *Applied Sciences*, v. 15, n. 8, 2025. doi: 10.3390/app15084425.
- [32] MIAO, X., OLIARO, G., ZHANG, Z., et al. “Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems”, *arXiv preprint arXiv:2312.15234*, 2024.
- [33] JIN, H., HAN, X., YANG, J., et al. “LLM Maybe LongLM: Self-Extend LLM Context Window Without Tuning”. In: *Proceedings of the 41st International Conference on Machine Learning*, 2024. arXiv:2401.01325.

- [34] LI, Z., LIU, Y., SU, Y., et al. “Prompt Compression for Large Language Models: A Survey”. 2025. arXiv:2410.12388.
- [35] JHA, S., ERDOGAN, L. E., KIM, S., et al. “Characterizing Prompt Compression Methods for Long Context Inference”. 2024. arXiv:2407.08892.
- [36] PAN, Z., WU, Q., JIANG, H., et al. “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression”. In: *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 963–981, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.57. Disponível em: <<https://aclanthology.org/2024.findings-acl.57/>>.
- [37] LISKAVETS, B., USHAKOV, M., ROY, S., et al. “Prompt Compression with Context-Aware Sentence Encoding for Fast and Improved LLM Inference”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, v. 39, pp. 24595–24604, 2025. doi: 10.1609/aaai.v39i23.34639. Disponível em: <<https://ojs.aaai.org/index.php/AAAI/article/view/34639>>.
- [38] LI, Y., DONG, B., GUERIN, F., et al. “Compressing Context to Enhance Inference Efficiency of Large Language Models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6342–6353, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.391. Disponível em: <<https://aclanthology.org/2023.emnlp-main.391/>>.
- [39] MICROSOFT RESEARCH. “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models”. GitHub repository, 2023. Disponível em: <<https://github.com/microsoft/LLMLingua>>. Acesso em 18 de julho de 2026.
- [40] MU, J., LI, X. L., GOODMAN, N. “Learning to Compress Prompts with Gist Tokens”. In: *Advances in Neural Information Processing Systems*, v. 36, 2023. arXiv:2304.08467.
- [41] CHEVALIER, A., WETTIG, A., AJITH, A., et al. “Adapting Language Models to Compress Contexts”. 2023. arXiv:2305.14788.
- [42] GE, T., HU, J., WANG, L., et al. “In-context Autoencoder for Context Compression in a Large Language Model”. 2023. arXiv:2307.06945.
- [43] WANG, C., YANG, Y., LI, R., et al. “Adapting LLMs for Efficient Context Processing through Soft Prompt Compression”. 2024. arXiv:2404.04997.

- [44] TAN, S., LI, X., PATIL, S., et al. “LLoCO: Learning Long Contexts Offline”. 2024. arXiv:2404.07979.
- [45] XU, F., SHI, W., CHOI, E. “RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation”. In: *The Twelfth International Conference on Learning Representations*, 2024. arXiv:2310.04408.
- [46] RAU, D., WANG, S., DÉJEAN, H., et al. “Context Embeddings for Efficient Answer Generation in RAG”. 2024. arXiv:2407.09252.
- [47] LOUIS, M., DÉJEAN, H., CLINCHANT, S. “PISCO: Pretty Simple Compression for Retrieval-Augmented Generation”. 2025. arXiv:2501.16075.
- [48] CHUANG, Y.-N., XING, T., CHANG, C.-Y., et al. “Learning to Compress Prompt in Natural Language Formats”. 2024. arXiv:2402.18700.
- [49] RATNER, N., LEVINE, Y., BELINKOV, Y., et al. “Parallel Context Windows for Large Language Models”. 2023. arXiv:2212.10947.
- [50] BAI, Y., LV, X., ZHANG, J., et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding”, *arXiv preprint arXiv:2308.14508*, 2023.
- [51] HSIEH, C.-P., SUN, S., KRIMAN, S., et al. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” *arXiv preprint arXiv:2404.06654*, 2024.
- [52] ZHANG, Z., LI, J., LAN, Y., et al. “An Empirical Study on Prompt Compression for Large Language Models”. 2025. arXiv:2505.00019.
- [53] ŁAJEWSKA, W., HARDALOV, M., AINA, L., et al. “Understanding and Improving Information Preservation in Prompt Compression for LLMs”. 2025. arXiv:2503.19114.
- [54] KAMRADT, G. “Needle In A Haystack – Pressure Testing LLMs”. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.
- [55] SHAHAM, U., IVGI, M., EFRAT, A., et al. “ZeroSCROLLS: A Zero-Shot Benchmark for Long Text Understanding”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7977–7989, 2023. arXiv:2305.14196.
- [56] YANG, Z., QI, P., ZHANG, S., et al. “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering”. In: *Proceedings of the 2018*

Conference on Empirical Methods in Natural Language Processing, pp. 2369–2380, 2018. doi: 10.18653/v1/D18-1259.

- [57] TRIVEDI, H., BALASUBRAMANIAN, N., KHOT, T., et al. “MuSiQue: Multihop Questions via Single-hop Question Composition”, *Transactions of the Association for Computational Linguistics*, v. 10, pp. 539–554, 2022. arXiv:2108.00573.
- [58] KWIATKOWSKI, T., PALOMAKI, J., REDFIELD, O., et al. “Natural Questions: A Benchmark for Question Answering Research”, *Transactions of the Association for Computational Linguistics*, v. 7, pp. 453–466, 2019.
- [59] JOSHI, M., CHOI, E., WELD, D. S., et al. “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1601–1611, 2017. arXiv:1705.03551.
- [60] ZHONG, M., YIN, D., YU, T., et al. “QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5905–5921, 2021. arXiv:2104.05938.
- [61] RADFORD, A., WU, J., CHILD, R., et al. *Language Models are Unsupervised Multitask Learners*. Relatório técnico, OpenAI, 2019. Disponível em: <<https://cdn.openai.com/better-language-models/language-models.pdf>>.
- [62] WORKDAY. “CPC: Official Implementation of Prompt Compression with Context-Aware Sentence Encoding”. GitHub repository, 2025. Disponível em: <<https://github.com/Workday/cpc>>. Acesso em 18 de julho de 2026.
- [63] SENTENCE TRANSFORMERS. “all-MiniLM-L6-v2”. Hugging Face model repository, 2022. Disponível em: <<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>>. Acesso em 18 de julho de 2026.